

**ÉCOLE SUPÉRIEURE DE TECHNOLOGIES ET DE MANAGEMENT
(ESTM)**

CAHIER DES CHARGES

Sopi

Plateforme numérique de gestion et de réservation
de services de proximité (Mécanique, Coiffure,
Pressing) à Keur Massar

Présenté par :
Abdoulaye Drame

Encadré par :
Dr El Hadji Bassirou TOURÉ
Enseignant-Chercheur en Informatique
DMI/FST/UCAD

Année Académique : 2024-2025

Fiche du document

Projet	Conception et réalisation d'une plateforme numérique de gestion et de réservation de services de proximité (Mécanique, Coiffure, Pressing) à Keur Massar
Nom de la plateforme	Sopi
Document	Cahier des charges – version 1.0
Auteur	Abdoulaye Drame – Licence en Informatique, ESTM
Encadrant	Dr El Hadji Bassirou TOURÉ, Enseignant-Chercheur en Informatique, DMI/FST/UCAD
Année académique	2024-2025
Contenu	Présentation du projet, analyse de l'existant, acteurs, 23 exigences fonctionnelles, 35 règles de gestion, exigences non fonctionnelles, modélisation UML (cas d'utilisation, séquences, classes), spécifications techniques (technologies, architecture, paiement DexPay, hébergement Dokploy, APK), sécurité, traçabilité, perspectives, glossaire

Table des matières

1. Présentation générale du projet	5
1.1 Contexte	5
1.2 Problématique	5
1.3 Objectifs du projet	6
1.3.1 Objectif général	6
1.3.2 Objectifs spécifiques	6
1.4 Présentation de la plateforme Sopi	6
1.5 Périmètre du projet	7
1.5.1 Dans le périmètre	7
1.5.2 Hors périmètre et perspectives	7
1.6 Livrables	7
2. Analyse et critique de l'existant	8
2.1 Méthodes actuelles de gestion	8
2.2 Solutions numériques existantes	8
2.3 Processus existants	8
2.4 Impacts négatifs des méthodes actuelles	9
2.5 Nécessité d'une solution informatique	9
3. Identification des acteurs	10
3.1 Acteurs primaires (utilisateurs directs)	10
3.2 Acteurs secondaires (systèmes externes)	10
4. Exigences fonctionnelles	11
5. Règles de gestion	14
6. Exigences non fonctionnelles	17
7. Modélisation fonctionnelle	18
7.1 Diagrammes de cas d'utilisation	18
7.1.1 Diagramme de cas d'utilisation général	18
7.1.2 Diagramme de cas d'utilisation « Client »	20
7.1.3 Diagramme de cas d'utilisation « Prestataire »	21
7.1.4 Diagramme de cas d'utilisation « Administrateur »	22
7.1.5 Inventaire des cas d'utilisation par acteur	23
7.2 Descriptions textuelles des cas d'utilisation	23
7.2.1 Cas commun : S'authentifier	23
7.2.2 Cas du Client	24
7.2.3 Cas du Prestataire	26
7.2.4 Cas de l'Administrateur	28
7.3 Diagrammes de séquence d'analyse	30
7.3.1 Client : Réserver un créneau	30
7.3.2 Prestataire : Traitement d'une demande	31
7.3.3 Client : Payer l'acompte	32
8. Modèle de données	34

8.1 Diagramme de classes d'analyse	34
8.2 Diagramme de classes de conception	37
9. Spécifications techniques	39
9.1 Technologies et outils	39
9.1.1 Langages de programmation	39
9.1.2 Frameworks et services	40
9.1.3 Outils de développement	41
9.2 Génération de l'application Android (APK)	42
9.3 Architecture technique	42
9.3.1 Vue d'ensemble : architecture en couches	43
9.3.2 Description des couches	43
9.3.3 Organisation modulaire de l'API	44
9.3.4 Cheminement d'une réservation	45
9.3.5 Cheminement du paiement et du retrait des gains	45
10. Sécurité et protection des données	47
11. Matrice de traçabilité	47
12. Contraintes, limites et perspectives	49
12.1 Contraintes et limites du projet	49
12.2 Perspectives	49
13. Glossaire	50

1. Présentation générale du projet

1.1 Contexte

Dans le département de Keur Massar — érigé en 46^e département du Sénégal par le décret n°2021-687 du 28 mai 2021 et peuplé de près de 760 000 habitants selon le recensement de 2023 — la sollicitation des services de proximité tels que les garages automobiles, les salons de coiffure ou les prestations de pressing repose traditionnellement sur le déplacement physique ou l'appel téléphonique improvisé. Actuellement, la prise de rendez-vous organisée via des outils numériques est quasi inexistante dans ces secteurs artisanaux.

Cette situation contraste avec le niveau d'équipement numérique du pays : selon le rapport trimestriel de l'Autorité de Régulation des Télécommunications et des Postes (ARTP), le taux de pénétration de l'Internet au Sénégal a atteint 112,42 % en 2024 (ce taux est rapporté au nombre d'abonnements et non au nombre de personnes connectées). Ce contraste entre la forte adoption du numérique et des pratiques informelles engendre des difficultés majeures pour les deux parties.

Du côté des **clients**, les déplacements vers les ateliers, les salons ou les comptoirs de pressing se font sans aucune garantie de disponibilité immédiate, ce qui provoque de longues files d'attente et une perte de temps considérable.

Du côté des **prestataires**, l'absence d'outils de planification entraîne une gestion artisanale de leur emploi du temps, causant une surcharge de travail par moments et une sous-utilisation de leurs capacités à d'autres périodes.

À cela s'ajoute le problème de l'**insécurité des engagements** : l'absence de confirmation formelle expose le prestataire au risque de rendez-vous non honorés (no-show), tandis que le client ne bénéficie d'aucune garantie quant à la prise en charge de sa demande. L'adoption massive des solutions de paiement mobile au Sénégal (Wave, Orange Money) offre désormais la possibilité d'instaurer un système d'acompte à la réservation, formalisant ainsi l'engagement mutuel des deux parties.

Il est primordial de s'attaquer à ces problèmes car la transformation numérique des services de proximité peut constituer un levier de développement économique pour les zones périurbaines comme Keur Massar, en formalisant une activité artisanale aujourd'hui largement informelle.

1.2 Problématique

La conception et le développement d'un système informatique centralisé permettant de moderniser la gestion des services de proximité à Keur Massar nécessite de résoudre plusieurs défis techniques liés à l'automatisation et à la centralisation des calendriers de multiples prestataires au sein d'une seule et même plateforme. Le défi majeur de ce projet consiste à transformer des processus de réservation manuels, dispersés et informels en un flux numérique structuré et unifié, afin d'optimiser le temps de travail des prestataires et d'éliminer les files d'attente pour les usagers. Pour relever ce défi global, la réalisation de cette plateforme mutualisée nécessite de résoudre les sous-problèmes techniques suivants :

- **Vérification et certification des profils** : mettre en place un processus de validation permettant à l'administrateur de vérifier les informations et les compétences de tous les prestataires candidats

(quelle que soit leur spécialité : mécanique, coiffure ou pressing) avant l'activation de leurs services sur la plateforme unique, afin de garantir la confiance et la sécurité des clients.

- **Gestion des conflits de réservations** : concevoir un système de gestion des calendriers permettant d'éviter que plusieurs clients ne réservent le même créneau horaire pour un même service.
- **Intégration de différents types de métiers** : concevoir une base de données flexible capable de gérer plusieurs catégories de services.
- **L'interactivité et le suivi transversal des prestations** : permettre au client de suivre en temps réel l'état de ses services auprès de différents prestataires via une interface unique, et mettre en place un système centralisé de notifications pour confirmer les rendez-vous, informer de la fin d'une prestation ou permettre le signalement de litiges, quelle que soit la nature du service réservé.
- **Engagement et paiement des réservations** : sécuriser l'engagement des deux parties en permettant le paiement en ligne d'un acompte par mobile money (Wave, Orange Money), afin de réduire les rendez-vous non honorés, et permettre au prestataire de récupérer simplement les gains issus de ses prestations.

1.3 Objectifs du projet

1.3.1 Objectif général

Concevoir et développer une plateforme numérique centralisée de gestion des agendas, de réservation, de paiement d'acompte et de notification pour les secteurs de la mécanique, de la coiffure et du pressing dans le département de Keur Massar, permettant aux clients d'accéder à l'ensemble de l'offre locale via une interface unique et aux prestataires de gérer leur activité au sein d'un écosystème mutualisé.

1.3.2 Objectifs spécifiques

1. Analyser et modéliser les processus métiers, le flux de validation des prestataires et le flux de paiement de l'acompte afin de produire les diagrammes UML.
2. Concevoir le schéma relationnel de la base de données et l'architecture logicielle intégrant le système de notification et la passerelle de paiement mobile.
3. Développer l'application mobile du client permettant de rechercher des prestataires par critères, de réserver des créneaux, de payer l'acompte en ligne et de consulter l'état de toutes ses réservations en cours via un tableau de bord unique.
4. Implémenter dans l'application mobile un espace prestataire permettant à chaque professionnel de gérer ses services, ses rendez-vous et son portefeuille (consultation du solde et retrait de ses gains) au sein de la plateforme commune.
5. Réaliser un back-office web d'administration permettant de superviser l'ensemble de la plateforme et de valider les comptes des nouveaux candidats.

1.4 Présentation de la plateforme Sopi

Sopi est la plateforme numérique qui répond à ces objectifs. Elle met en relation, au sein d'un seul système, les clients et les prestataires locaux des trois métiers ciblés (mécanique, coiffure, pressing) du département de Keur Massar. Elle se compose de :

- une **application mobile** (Flutter), commune aux clients et aux prestataires, livrée sous forme de fichier APK téléchargeable et installable sur les téléphones Android ;

- un **back-office web** (React) destiné à l'administrateur ;
- une **API REST** (Laravel) qui porte toutes les règles métier ;
- une **base de données relationnelle** (PostgreSQL) ;
- des **services de notification** (e-mail et notifications push) ;
- la **passerelle de paiement mobile DexPay** (Wave, Orange Money), utilisée en mode test ;
- un **hébergement géré par Dokploy** pour le backend, le back-office et la base de données.

1.5 Périmètre du projet

1.5.1 Dans le périmètre

- Recherche de prestataires de mécanique, coiffure et pressing à Keur Massar, avec filtres par quartier et disponibilité ;
- gestion des agendas, verrouillage atomique des créneaux, réservation soumise à l'acceptation du prestataire ;
- paiement en ligne d'un acompte de 30 % par Wave ou Orange Money (DexPay), code de réservation, commission de la plateforme, portefeuille et retrait des gains du prestataire ;
- validation des prestataires, avis, litiges, sanctions et modération par l'administrateur ;
- notifications par e-mail et par notification push (l'administrateur est alerté par des bannières sur son tableau de bord) ;
- application mobile Android (APK) pour les clients et les prestataires ; back-office web pour l'administrateur ;
- hébergement du backend, du back-office et de la base de données avec Dokploy, en HTTPS.

1.5.2 Hors périmètre et perspectives

- publication de l'application sur une boutique d'applications (Google Play), envisagée comme perspective ;
- encaissement et versement d'argent réel : DexPay est utilisée en mode test (sandbox), les transactions sont simulées ;
- changement de prestataire de paiement ou ajout d'un nouveau canal de notification : rendus possibles par les interfaces PaiementService et NotificationService, sans modification des règles métier ;
- ajout de nouveaux types de services ou de nouveaux quartiers : prévu par la maintenabilité du code.

1.6 Livrables

- le **fichier APK** de l'application mobile (client et prestataire), obtenu avec la commande `flutter build apk --release` ;
- le **back-office web** d'administration ;
- l'**API REST** Laravel, documentée par Swagger ou Postman ;
- la **base de données** PostgreSQL correspondant au diagramme de classes de conception ;
- les **diagrammes UML** (cas d'utilisation, séquences, classes d'analyse et de conception) et les schémas d'architecture ;
- le **mémoire** de Licence en Informatique et le présent cahier des charges.

2. Analyse et critique de l'existant

2.1 Méthodes actuelles de gestion

Dans le département de Keur Massar, la sollicitation des services de proximité tels que les garages automobiles, les salons de coiffure ou les prestations de pressing repose traditionnellement sur des démarches spontanées. L'interaction entre l'artisan et son client s'établit suivant deux canaux principaux : le déplacement physique improvisé vers les ateliers ou l'appel téléphonique direct de dernière minute.

Du côté des prestataires, la gestion opérationnelle de l'activité est purement artisanale et informelle. Les professionnels n'utilisent aucun outil numérique ni calendrier partagé pour planifier leurs journées. Le suivi des demandes et de l'emploi du temps repose uniquement sur la mémoire de l'artisan ou sur des supports physiques rudimentaires tels que des registres papier, des cahiers de bord ou des carnets de notes classiques. Le règlement de la prestation se fait le plus souvent en espèces ou par mobile money personnel, directement entre le client et l'artisan, une fois le travail terminé, sans trace ni engagement préalable.

2.2 Solutions numériques existantes

Il convient de noter que des solutions numériques existent déjà sur des segments voisins, sans couvrir le périmètre visé. Des outils de messagerie généralistes comme **WhatsApp Business** sont parfois utilisés par les artisans les plus équipés pour échanger avec leurs clients, mais sans gestion structurée d'agenda ni verrouillage des créneaux. Par ailleurs, des startups sénégalaises comme **Niafey** proposent une application de réservation de services à domicile (coiffure, plomberie, électricité, chauffeurs privés), mais selon un modèle d'intervention chez le client plutôt qu'un rendez-vous en atelier, et sans couvrir la mécanique automobile ni le pressing. Aucune solution recensée ne propose donc, à ce jour, une gestion centralisée des rendez-vous en atelier pour les trois corps de métier ciblés par Sopi à Keur Massar.

2.3 Processus existants

Le déroulement étape par étape des pratiques actuelles met en évidence des ruptures d'organisation majeures à chaque phase clé de l'activité :

Tableau 1 – Analyse des processus actuels

Processus	Méthode actuelle	Difficultés identifiées
Demande de service et prise de rendez-vous	Le client appelle de manière spontanée ou se déplace directement sur site au sein de l'atelier sans planification préalable.	Aucune garantie de disponibilité immédiate du prestataire, obligeant le client à faire face à l'imprévu.
Gestion de l'agenda de l'artisan	L'artisan planifie ses travaux de tête ou note les demandes de manière séquentielle sur un carnet en papier.	Risques élevés d'oublis, de mauvaise gestion des périodes d'inactivité et de chevauchements d'horaires pour un même service.
Suivi de l'état des prestations	Le client doit patienter longtemps sur place ou appeler l'artisan à plusieurs reprises pour s'informer de l'avancement.	Absence totale de visibilité à distance, provoquant de longues files d'attente et l'engorgement des ateliers.

Processus	Méthode actuelle	Difficultés identifiées
Engagement et paiement	Aucun engagement préalable : le client règle le montant en espèces ou par mobile money personnel à la fin de la prestation.	Rendez-vous non honorés sans conséquence, créneaux perdus pour l'artisan, aucune trace de la transaction.

2.4 Impacts négatifs des méthodes actuelles

Ces méthodes purement manuelles génèrent des impacts négatifs directs et quantifiables sur l'écosystème de Keur Massar :

- **Perte de temps considérable pour les clients** : les déplacements vers les ateliers ou les salons de coiffure se font sans certitude. Cela engendre de longues files d'attente sur place et immobilise le temps des usagers de manière stérile.
- **Surcharge de travail et sous-utilisation des capacités** : l'absence d'outils de planification entraîne une gestion artisanale du temps de travail. Les artisans subissent une forte surcharge par moments (comme les week-ends) et une sous-utilisation critique de leurs capacités à d'autres périodes de la semaine.
- **Risques de conflits d'agendas et d'erreurs** : le suivi basé sur la mémoire ou sur papier cause des erreurs fréquentes de planification. Plusieurs clients peuvent se voir attribuer verbalement le même créneau horaire, ce qui nuit à la confiance indispensable entre l'artisan et l'utilisateur.
- **Rendez-vous non honorés et manque à gagner** : faute d'engagement préalable, un client peut ne pas se présenter sans prévenir. Le créneau est alors perdu pour l'artisan, qui ne peut ni le proposer à un autre client ni être compensé.

2.5 Nécessité d'une solution informatique

La transformation numérique via la plateforme centralisée Sopi est techniquement indispensable pour surmonter ces difficultés :

1. **Gestion automatisée des accès concurrents** : seul un système informatique centralisé est capable de gérer en temps réel les calendriers de multiples prestataires afin d'interdire que plusieurs clients ne réservent le même créneau horaire pour un même service.
2. **Flexibilité et centralisation des données** : l'application permet de concevoir une structure capable de gérer sous une interface unique des catégories de métiers différentes (mécanique, coiffure, pressing), offrant une vue d'ensemble transversale.
3. **Asynchronisme et suivi en temps réel** : l'implémentation d'un système centralisé de notifications (confirmations, fins de prestation) libère l'utilisateur de l'attente physique et fluidifie la communication de manière instantanée.
4. **Engagement et paiement sécurisé** : le paiement en ligne d'un acompte par mobile money formalise l'engagement du client, réduit les rendez-vous non honorés et laisse une trace de chaque transaction. Un portefeuille permet au prestataire de suivre ses gains et de les retirer simplement.

3. Identification des acteurs

Pour concevoir les cas d'utilisation du système de manière rigoureuse, il est impératif d'identifier les entités qui interagiront directement ou indirectement avec la plateforme numérique.

3.1 Acteurs primaires (utilisateurs directs)

Les acteurs primaires sont les utilisateurs humains qui interagissent directement avec l'interface de l'application pour satisfaire leurs propres besoins métiers :

Tableau 2 – Acteurs primaires

Acteur	Interface	Rôle dans la plateforme
Client (ou usager)	Application mobile	Recherche des professionnels à Keur Massar par critères, réserve des créneaux horaires disponibles, paie l'acompte de ses réservations acceptées et suit en temps réel l'état de l'ensemble de ses réservations via un tableau de bord unique.
Prestataire (artisan)	Application mobile (espace dédié)	Professionnel de la mécanique, de la coiffure ou du pressing. Configure son catalogue de services, déclare ses disponibilités horaires, gère ses rendez-vous, met à jour l'état d'avancement de ses prestations et consulte son portefeuille pour retirer ses gains.
Administrateur	Back-office web	Gestionnaire central du système. Supervise l'ensemble de la plateforme et pilote le processus de vérification et de certification des profils afin de valider les informations et compétences des prestataires candidats avant l'activation de leurs services.

3.2 Acteurs secondaires (systèmes externes)

Les acteurs secondaires sont des entités externes sollicitées par le système pour compléter l'exécution de certains processus automatisés :

Tableau 3 – Acteurs secondaires

Acteur	Rôle dans la plateforme
Service de notification	Composant technologique externe sollicité par l'application pour acheminer aux clients et aux prestataires, par notification push ou par e-mail, les messages liés au cycle de vie de la plateforme : confirmation d'un rendez-vous, demande de paiement de l'acompte, fin d'une prestation, résultat de l'examen d'un dossier, ou décision rendue sur un litige préalablement signalé par un client ou un prestataire.
Prestataire de paiement (DexPay)	Plateforme externe d'agrégation de paiements mobiles (Wave, Orange Money), sollicitée par l'application pour encaisser l'acompte du client et pour verser au prestataire les gains qu'il retire de son portefeuille. Il confirme chaque opération à la plateforme par un message automatique. Dans le cadre de ce projet, il est utilisé dans son environnement de test (sandbox).

4. Exigences fonctionnelles

Les exigences fonctionnelles définissent de manière précise et non ambiguë les capacités opérationnelles ainsi que les services métiers que la plateforme Sopi doit posséder pour répondre aux besoins de ses utilisateurs. Elles sont identifiées par un code RF01 à RF23 et classées par pôle d'activité ; la priorité est indiquée pour chacune.

Tableau 4 – Spécifications des exigences fonctionnelles par pôle d'activité

ID	Description de l'exigence	Priorité
Pôle Client		
RF01	Le système doit permettre au client de créer et gérer son profil.	Obligatoire
RF02	Le système doit permettre au client d'effectuer des recherches de prestataires à Keur Massar par domaine de métier (mécanique, coiffure, pressing) et de filtrer les résultats par quartier ou disponibilité.	Obligatoire
RF03	Le système doit afficher l'agenda d'un prestataire en temps réel et verrouiller temporairement un créneau sélectionné pendant le processus de réservation afin d'éviter tout conflit d'accès concurrents.	Obligatoire
RF04	Le système doit permettre au client de gérer ses réservations depuis un tableau de bord utilisateur centralisé, incluant la validation, la consultation du suivi (états, historique, détails) ainsi que l'annulation d'une réservation selon les conditions de délais définies par l'application. L'acompte déjà payé n'est pas remboursé en cas d'annulation par le client.	Obligatoire
RF05	Le système doit permettre au client de consulter le profil public d'un prestataire, incluant son catalogue de services, ses tarifs, les photos de son atelier ainsi que les évaluations laissées par les autres clients.	Obligatoire
RF06	Le système doit permettre au client de soumettre une note chiffrée et un avis textuel sur le profil du prestataire uniquement après la clôture d'une prestation (réservation « Terminée »).	Importante
RF07	Le système doit permettre au client de signaler un litige depuis l'historique d'une réservation, avec possibilité de fournir une description détaillée et des preuves photographiques.	Obligatoire
Pôle Prestataire		
RF08	Le système doit permettre au prestataire de soumettre un dossier d'inscription complet incluant ses justificatifs d'identité et professionnels (CNI, diplôme ou attestation), et de suivre l'état d'avancement de sa validation.	Obligatoire
RF09	Le système doit permettre au prestataire de gérer son catalogue de services et ses disponibilités horaires, incluant la configuration des prestations, des tarifs, des durées estimées et des créneaux de disponibilité.	Obligatoire
RF10	Le système doit fournir au prestataire un tableau de bord d'activité affichant son planning de rendez-vous, les demandes de réservation en attente ainsi que les indicateurs liés à son activité, et lui permettre d'accepter ou de refuser les demandes de réservation dans un délai de 12 heures, passé lequel la demande expire et le créneau est libéré. L'acceptation fait passer la réservation à « Acceptée, en attente de paiement ».	Obligatoire
RF11	Le système doit permettre au prestataire de mettre à jour en temps réel l'état d'avancement d'une prestation en faisant passer la réservation de « Confirmée » à « En cours » sur saisie du code de réservation présenté par le client, puis à « Terminée ».	Obligatoire

ID	Description de l'exigence	Priorité
RF12	Le système doit permettre au prestataire de déclarer des plages d'indisponibilité temporaire (congé ou urgences), bloquant instantanément toute nouvelle réservation client sur ces créneaux. Les réservations déjà enregistrées sur la plage sont annulées après confirmation du prestataire, leurs acomptes éventuellement payés sont remboursés et les clients concernés en sont notifiés.	Importante
RF13	Le système doit notifier automatiquement le prestataire en cas de litige ouvert à son encontre et lui mettre à disposition une interface pour soumettre sa version des faits à l'administration.	Importante
Pôle Administrateur		
RF14	Le système doit permettre à l'administrateur d'accéder de manière sécurisée à l'espace d'administration afin de superviser et gérer les différentes activités de la plateforme.	Obligatoire
RF15	Le système doit permettre à l'administrateur de gérer la validation des comptes prestataires, incluant la consultation des justificatifs professionnels, l'acceptation ou le refus des demandes d'inscription.	Obligatoire
RF16	Le système doit permettre à l'administrateur de consulter son tableau de bord afin de superviser l'activité globale de la plateforme : nombre d'utilisateurs et de prestataires, statistiques d'activité (réservations par statut, par période et par domaine de métier), revenus de la plateforme (acomptes encaissés, commission retenue, montants retirés), ainsi que les éléments en attente de traitement (dossiers d'inscription, litiges ouverts, avis signalés) depuis lesquels il accède aux fonctions correspondantes.	Obligatoire
RF17	Le système doit permettre à l'administrateur de gérer les litiges signalés sur la plateforme en consultant les informations associées et en enregistrant une décision finale parmi : rejet du signalement, avertissement, suspension temporaire ou bannissement définitif. La décision est notifiée aux parties et, lorsqu'elle constitue une sanction, appliquée conformément à RF19.	Obligatoire
RF18	Le système doit orchestrer l'envoi des notifications automatiques (push et e-mail) à destination des clients et des prestataires à chaque étape clé du cycle de vie du système (validation de compte, acceptation d'une demande en attente de paiement, confirmation de paiement, alertes de rendez-vous, traitement de signalement).	Obligatoire
RF19	Le système doit permettre à l'administrateur de gérer les sanctions appliquées aux utilisateurs et prestataires, incluant l'avertissement, la suspension temporaire ainsi que le bannissement définitif, avec conservation de l'historique des sanctions afin de permettre le suivi des récidives.	Obligatoire
RF20	Le système doit permettre à l'administrateur de modérer les avis signalés par les prestataires, en décidant de les supprimer ou de les conserver. La décision est tracée (administrateur, date) et notifiée au prestataire.	Importante
Pôle Paiement		
RF21	Le système doit permettre au client de payer en ligne un acompte de 30 % du tarif de la prestation, via un prestataire de paiement mobile (Wave, Orange Money), après acceptation de sa demande par le prestataire et dans un délai de 2 heures. Le paiement confirmé fait passer la réservation à « Confirmée » et génère un code de réservation ; à défaut de paiement dans le délai, la réservation est annulée et le créneau libéré. Si le prestataire annule une réservation dont l'acompte a été payé, celui-ci est remboursé intégralement.	Obligatoire

ID	Description de l'exigence	Priorité
RF22	Le système doit répartir l'acompte à la clôture de la prestation (réservation « Terminée ») : une commission de la plateforme, calculée selon un taux paramétrable (10 % du tarif par défaut), est retenue et le reste de l'acompte est crédité au portefeuille du prestataire. Le solde de la prestation est réglé directement entre le client et le prestataire.	Obligatoire
RF23	Le système doit permettre au prestataire de consulter son portefeuille (solde disponible, historique des gains et des retraits) et de retirer son solde vers son compte mobile money, chaque retrait étant tracé et son résultat notifié au prestataire.	Obligatoire

Synthèse : 23 exigences fonctionnelles, dont 19 de priorité « Obligatoire » (RF01 à RF05, RF07 à RF11, RF14 à RF19 et RF21 à RF23) et 4 de priorité « Importante » (RF06, RF12, RF13 et RF20).

5. Règles de gestion

Les règles de gestion ci-dessous rassemblent, par thème, les règles métier qui découlent des exigences et des cas d'utilisation du mémoire. Elles s'appliquent dans la couche métier (API Laravel).

Tableau 5 – Règles de gestion de la plateforme

ID	Règle	Exigences liées
Comptes et accès		
RG01	Le statut d'un compte est l'un des suivants : En attente de validation, Actif, Refusé, Suspendu, Banni. Seul un compte « Actif » peut s'authentifier ; un compte suspendu ou banni ne peut plus s'authentifier et le motif de la sanction est affiché.	RF14, RF19
RG02	Après cinq échecs d'authentification consécutifs, le compte est bloqué temporairement et la procédure « Mot de passe oublié » est proposée.	RF14
RG03	Les mots de passe sont hachés avec Bcrypt. L'accès aux routes protégées exige une authentification et le respect du rôle de l'utilisateur (un client ne peut pas appeler une route réservée à l'administrateur).	RF14
RG04	Un prestataire n'apparaît dans les recherches et ne peut être réservé qu'une fois son compte « Actif » (dossier validé par l'administrateur).	RF02, RF15
Validation des prestataires		
RG05	Le dossier d'inscription comprend les informations du prestataire (nom de la structure, métier, adresse à Keur Massar, identifiants) et ses justificatifs (carte d'identité, diplôme ou attestation) au format PDF ou image. Un document manquant ou d'un format invalide bloque la soumission.	RF08
RG06	À la soumission, le compte est créé au statut « En attente de validation » et un lien de suivi personnel est envoyé par e-mail ; le prestataire peut suivre l'état de son dossier (En attente, Actif, Refusé) sans s'authentifier, avec le motif en cas de refus.	RF08
RG07	L'administrateur accepte le dossier (compte « Actif ») ou le refuse en saisissant un motif (compte « Refusé ») ; le prestataire est notifié par e-mail dans les deux cas. Un dossier aux justificatifs illisibles ou incomplets peut rester « En attente ».	RF15
Agenda et réservation		
RG08	Lorsqu'un client sélectionne un créneau libre, le système le verrouille temporairement de façon atomique dans la base de données ; si deux clients sélectionnent le même créneau au même instant, un seul obtient le verrou.	RF03
RG09	Si le créneau est déjà verrouillé ou réservé, la sélection est refusée, l'agenda est actualisé et un message d'erreur est affiché. Si le client confirme après l'expiration du verrou, le créneau est libéré et le message « Délai dépassé » est affiché ; si le client abandonne, le verrou expire de lui-même.	RF03
RG10	À la confirmation par le client dans le délai du verrou, la réservation est enregistrée au statut « En attente » ; aucun paiement n'est demandé à ce stade. Le prestataire est notifié sans que l'identité complète du client lui soit révélée.	RF03, RF18
RG11	Le prestataire dispose de 12 heures pour répondre. Sans réponse, la réservation passe à « Annulée », le créneau est libéré et le client est informé.	RF10
RG12	Le refus d'une demande exige un motif : la réservation passe à « Refusée », le créneau est libéré et le client est notifié avec le motif.	RF10

ID	Règle	Exigences liées
RG13	L'acceptation fait passer la réservation à « Acceptée, en attente de paiement » ; le créneau reste bloqué et le client est notifié qu'il dispose de 2 heures pour payer l'acompte.	RF10, RF21
RG14	Le cycle de vie d'une réservation est : En attente → Acceptée (en attente de paiement) → Confirmée → En cours → Terminée, avec les issues Refusée et Annulée. L'enchaînement des statuts est contrôlé par le système.	RF04, RF10, RF11
RG15	Le client peut annuler une réservation selon les conditions de délais définies par l'application ; l'acompte déjà payé n'est pas remboursé en cas d'annulation par le client.	RF04
Paiement, commission et portefeuille		
RG16	L'acompte est égal à 30 % du tarif de la prestation. Il est payable en ligne par Wave ou Orange Money, via une session de paiement DexPay, dans un délai de 2 heures après l'acceptation.	RF21
RG17	L'application ne manipule jamais de numéro de carte ni de code secret : le client est redirigé vers la page de paiement hébergée par DexPay.	RF21
RG18	Le résultat du paiement revient au système par un webhook signé (HMAC-SHA256) dont l'authenticité est vérifiée avant toute mise à jour de la réservation.	RF21
RG19	Un paiement a l'un des statuts : EN_ATTENTE, REUSSI, ECHOUE, REMBOURSE. En cas d'échec, le paiement est enregistré « ECHOUE », la réservation reste « Acceptée, en attente de paiement » et le client peut réessayer tant que le délai de 2 heures n'est pas écoulé.	RF21
RG20	Sans paiement dans le délai de 2 heures, la réservation passe à « Annulée », le créneau est libéré, l'action « Payer l'acompte » n'est plus proposée et le client est informé.	RF21
RG21	Un paiement réussi fait passer la réservation à « Confirmée » et génère un code de réservation, transmis au client par notification et affiché dans l'application ; le prestataire est informé que la réservation est confirmée.	RF21
RG22	Le jour du rendez-vous, le prestataire saisit le code de réservation présenté par le client : la réservation passe de « Confirmée » à « En cours », puis à « Terminée ». Un code invalide est refusé et la saisie est réaffichée.	RF11
RG23	Au passage à « Terminée », la commission de la plateforme (taux paramétrable, 10 % du tarif par défaut) est retenue et le reste de l'acompte (acompte moins commission) est crédité au portefeuille du prestataire. Avec les taux par défaut, le prestataire reçoit donc 20 % du tarif dans son portefeuille.	RF22
RG24	Le solde de la prestation (70 % du tarif) est réglé directement entre le client et le prestataire, sur place.	RF22
RG25	Si le prestataire annule une réservation dont l'acompte a été payé (annulation ou indisponibilité), l'acompte est remboursé intégralement (paiement « REMBOURSE ») et le client est notifié.	RF12, RF21
Indisponibilités		
RG26	Lorsque le prestataire déclare une plage d'indisponibilité, les créneaux libres de la plage sont bloqués immédiatement. Les réservations déjà enregistrées sur la plage lui sont présentées : après sa confirmation elles passent à « Annulée », leurs acomptes payés sont remboursés et les clients sont notifiés ; s'il y renonce, seuls les créneaux libres sont bloqués et les réservations existantes restent inchangées.	RF12
Retrait des gains		

ID	Règle	Exigences liées
RG27	Le prestataire peut retirer son solde disponible vers son compte mobile money (Wave ou Orange Money). Le montant demandé ne peut dépasser le solde disponible ni sortir des limites fixées par l'opérateur.	RF23
RG28	Un retrait a l'un des statuts : EN_COURS, REUSSI, ECHOUE. Le système débite le portefeuille, enregistre le retrait « En cours » et demande le virement à DexPay ; le résultat met à jour l'historique. En cas d'échec, le retrait est « Échoué », le montant est remis dans le solde disponible et le prestataire est informé.	RF23
RG29	Les opérations financières (enregistrement d'un paiement, crédit du portefeuille, débit lors d'un retrait) sont exécutées dans des transactions de base de données afin que le solde reste cohérent.	RF22, RF23
Avis, litiges et sanctions		
RG30	Un client ne peut déposer une note chiffrée et un avis textuel que sur une réservation « Terminée ». Un prestataire peut signaler un avis ; l'administrateur le supprime ou le conserve, et la décision est tracée (administrateur, date) et notifiée.	RF06, RF20
RG31	Un client peut signaler un litige depuis l'historique d'une réservation, avec une description détaillée et des preuves photographiques. Le prestataire concerné est notifié et peut soumettre sa version des faits.	RF07, RF13
RG32	Un litige est « Ouvert » puis « Clôturé » par la décision de l'administrateur : rejet du signalement (aucune sanction), avertissement, suspension temporaire ou bannissement définitif. Un litige clôturé n'accepte pas de nouvelle décision.	RF17
RG33	Chaque sanction est enregistrée dans l'historique des sanctions de l'utilisateur (client ou prestataire), consultable par l'administrateur pour suivre les récidives, et ses effets sont appliqués au compte.	RF17, RF19
Notifications		
RG34	Les notifications sont envoyées par e-mail (Laravel Mail, SMTP) et par notification push (Firebase Cloud Messaging) aux étapes clés : résultat de l'examen d'un dossier, nouvelle demande, acceptation ou refus, confirmation de paiement avec code, mise à jour de la prestation, décision sur un litige ou un avis signalé.	RF18
RG35	L'administrateur ne reçoit ni e-mail ni notification push : les éléments à traiter (dossiers en attente, litiges ouverts, avis signalés) lui sont signalés par des bannières d'alerte sur le tableau de bord ; aucune bannière n'est affichée s'il n'y a rien à traiter.	RF16, RF18

6. Exigences non fonctionnelles

Les exigences non fonctionnelles décrivent la manière dont le système doit se comporter.

Tableau 6 – Spécifications des exigences non fonctionnelles

Catégorie	Exigence	Critère de mesure
Performance	Le système doit répondre rapidement aux requêtes de recherche et de réservation, même en cas de faible débit internet.	Temps de réponse moyen < 2 secondes pour les requêtes API ; taille des images compressée sous 500 Ko.
Sécurité	Le système doit garantir l'authentification stricte des utilisateurs et la confidentialité des échanges lors des litiges.	Utilisation du hachage Bcrypt pour les mots de passe ; masquage de l'identité du client dans les notifications au prestataire.
Utilisabilité	Le système doit offrir une interface intuitive permettant à un utilisateur novice de réserver un service facilement.	Réalisation d'une réservation en moins de 4 clics ; interface mobile ergonomique, utilisable à une main.
Fiabilité	Le système doit être disponible de manière permanente pour assurer la continuité des services de proximité.	Taux de disponibilité (uptime) de 99 % ; sauvegarde automatique de la base de données toutes les 24 h.
Portabilité	Le système doit fonctionner sur les téléphones Android (et iOS) pour les clients et les prestataires, et sur les navigateurs récents pour l'administrateur.	Application mobile compatible Android et iOS ; back-office web compatible avec Chrome, Firefox et Edge.
Maintenabilité	Le code doit être structuré de manière à faciliter l'ajout de nouveaux types de services ou de quartiers.	Architecture MVC respectée ; documentation des routes API via Swagger ou Postman.

7. Modélisation fonctionnelle

Après avoir spécifié les exigences fonctionnelles et non fonctionnelles, l'étape suivante consiste à les traduire en une représentation formelle et structurée du système. Le Langage de Modélisation Unifié (UML) offre un ensemble de notations graphiques standardisées qui permettent de modéliser un système logiciel sous différents angles. Trois types de diagrammes UML complémentaires sont utilisés :

- **Le diagramme de cas d'utilisation** répond à la question « Quoi ? » : il définit le périmètre fonctionnel du système en décrivant les interactions et les objectifs de chaque acteur (Client, Prestataire, Administrateur) avec la plateforme.
- **Le diagramme de séquence** répond à la question « Comment et quand ? » : il illustre l'ordre chronologique des messages échangés entre les différents objets ou composants du système pour réaliser une fonctionnalité précise.
- **Le diagramme de classes** répond à la question « Avec quoi ? » : il décrit la structure statique du système en représentant les concepts métier (Utilisateur, Service, Réservation, Sanction...), leurs attributs et les relations qui les lient. Il est fondamental pour la conception de la base de données relationnelle et l'organisation du code source.

7.1 Diagrammes de cas d'utilisation

La démarche adoptée est itérative : à partir de la liste détaillée des fonctionnalités, on aboutit à un ensemble de cas d'utilisation principaux reflétant les objectifs métier des utilisateurs. La structure repose sur une vue globale (diagramme général) et des vues détaillées par acteur, intégrant la généralisation et les relations « include » et « extend ».

7.1.1 Diagramme de cas d'utilisation général

Ce diagramme offre une vue d'ensemble des acteurs et des objectifs principaux qu'ils peuvent atteindre dans le système. Le concept d'héritage y est appliqué pour factoriser les fonctionnalités communes à tous, telles que « Gérer son profil » et « S'authentifier ». Il cartographie tous les acteurs identifiés (principaux et secondaires : le Service de notification et le Prestataire de paiement DexPay) en les reliant aux grands blocs fonctionnels de la plateforme Sopi. « S'authentifier » est un préalable (include) de tous les cas d'utilisation, à l'exception de « S'inscrire » et de « Soumettre un dossier d'inscription ».

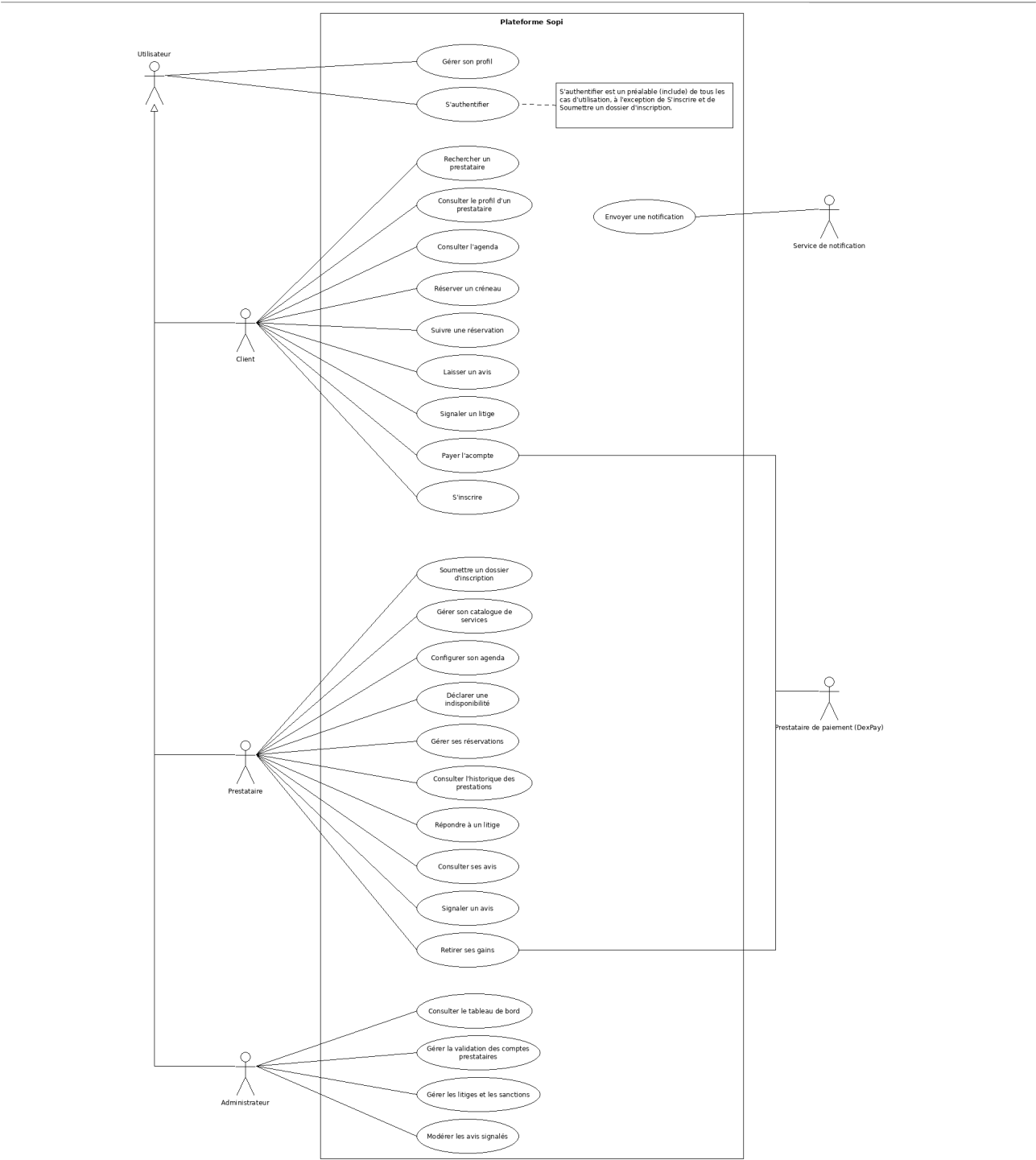


Figure 1 – Diagramme de cas d'utilisation général de la plateforme Sopi

7.1.2 Diagramme de cas d'utilisation « Client »

Le diagramme suivant détaille les cas d'utilisation spécifiques à l'acteur Client : la recherche de prestataires à Keur Massar (mécanique, coiffure, pressing), la réservation de créneaux, le suivi de ses réservations, le paiement de l'acompte après acceptation de sa demande, la notation des prestations (avis) ainsi que la déclaration de litiges.

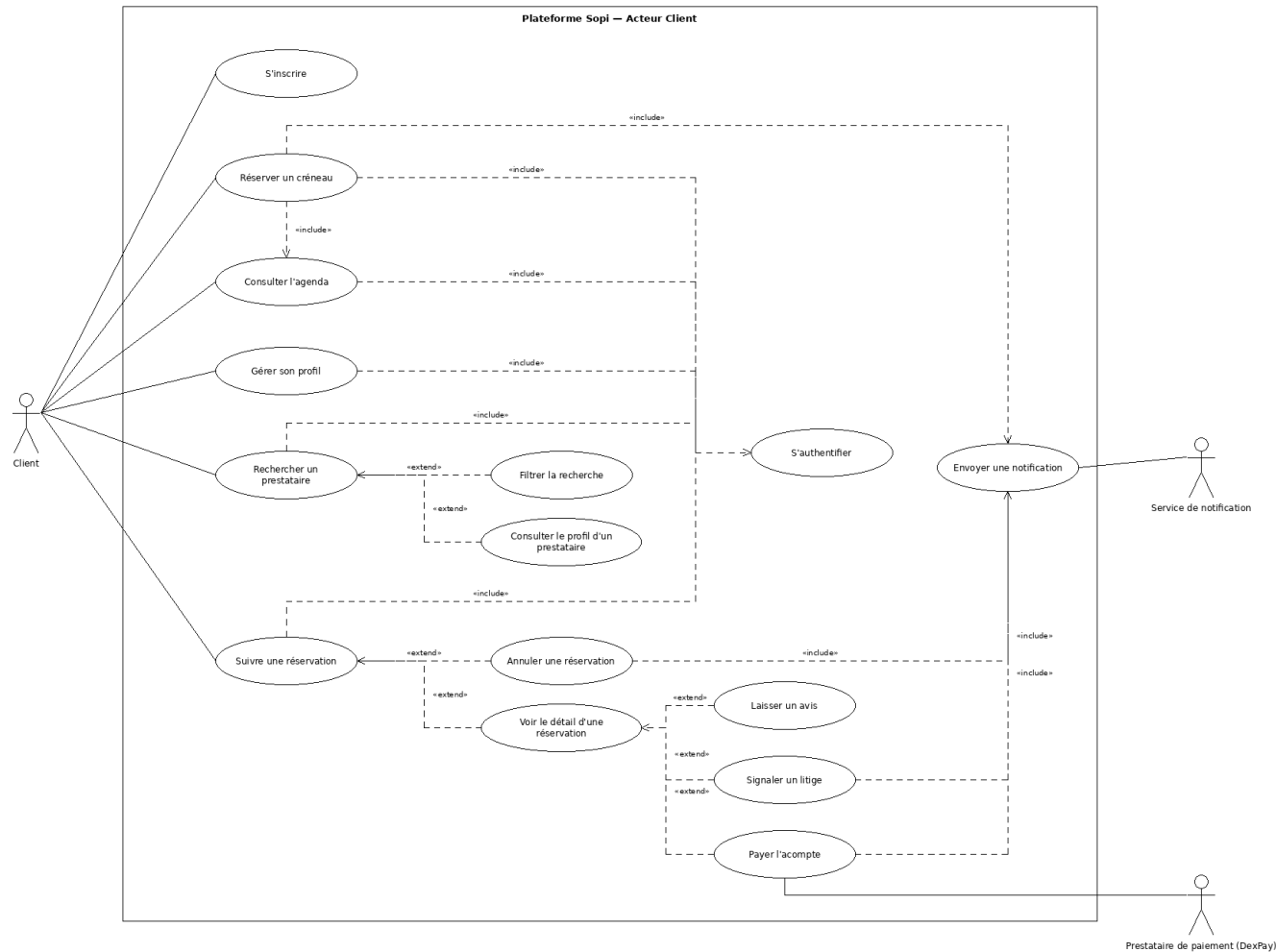


Figure 2 – Diagramme de cas d'utilisation détaillé — Acteur Client

7.1.3 Diagramme de cas d'utilisation « Prestataire »

Le diagramme suivant détaille les actions et objectifs professionnels du Prestataire : la gestion de son profil, de son catalogue et de ses tarifs, la configuration et la mise à jour de son agenda (créneaux horaires), le traitement des demandes de réservation et le suivi des prestations, le retrait de ses gains issus des acomptes, ainsi que le traitement des litiges.

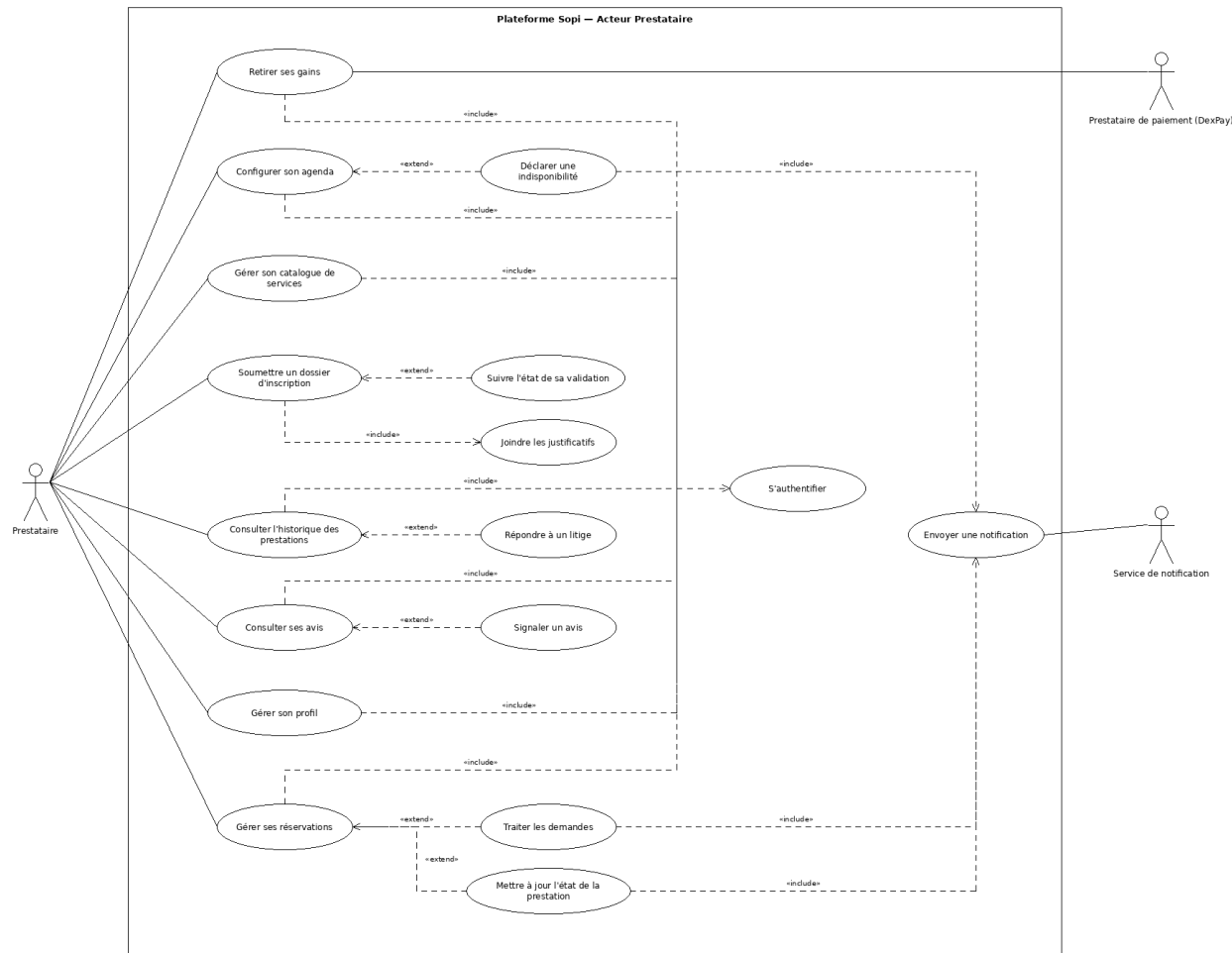


Figure 3 – Diagramme de cas d'utilisation détaillé — Acteur Prestataire

7.1.4 Diagramme de cas d'utilisation « Administrateur »

L'action centrale est « Gérer la validation des comptes prestataires », essentielle pour garantir la fiabilité des artisans dans le département de Keur Massar. L'administrateur est également chargé de la supervision de la plateforme, à travers son tableau de bord (indicateurs, revenus et alertes), et de la résolution des litiges. Toutes ces actions de gouvernance nécessitent obligatoirement une authentification préalable via le cas « S'authentifier ».

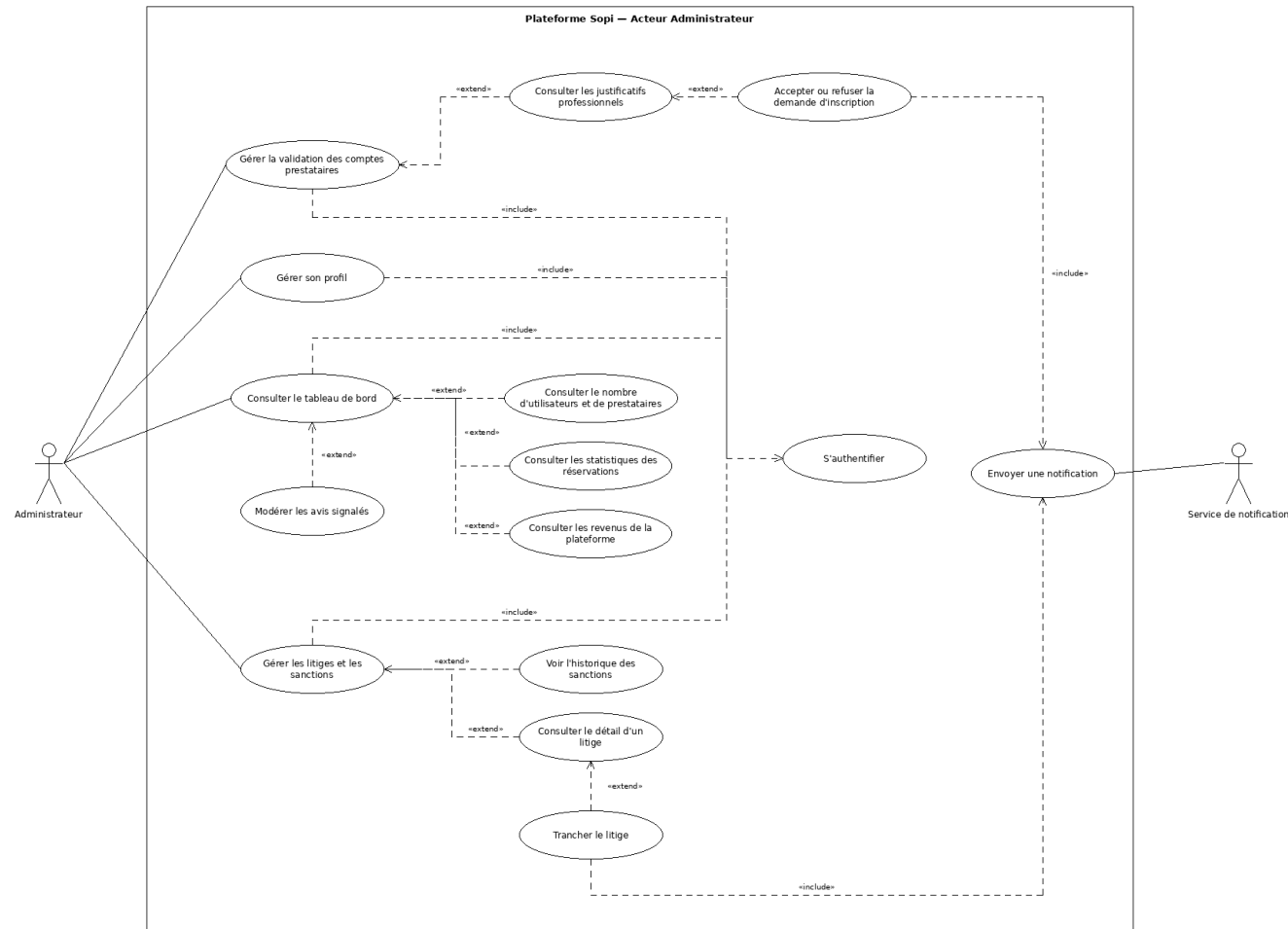


Figure 4 – Diagramme de cas d'utilisation détaillé — Acteur Administrateur

7.1.5 Inventaire des cas d'utilisation par acteur

Tableau 7 – Inventaire des cas d'utilisation par acteur

Acteur	Cas d'utilisation	Exigences
Utilisateur (Client, Prestataire, Administrateur)	S'authentifier ; Gérer son profil	RF01, RF14
Client	S'inscrire ; Rechercher un prestataire (étendu par Filtrer la recherche et Consulter le profil d'un prestataire) ; Consulter l'agenda ; Réserver un créneau ; Suivre une réservation (étendu par Annuler une réservation et Voir le détail d'une réservation) ; Payer l'acompte ; Laisser un avis ; Signaler un litige	RF01 à RF07, RF21
Prestataire	Soumettre un dossier d'inscription (avec Joindre les justificatifs et Suivre l'état de sa validation) ; Gérer son catalogue de services ; Configurer son agenda (étendu par Déclarer une indisponibilité) ; Gérer ses réservations (étendu par Traiter les demandes et Mettre à jour l'état de la prestation) ; Consulter l'historique des prestations (étendu par Répondre à un litige) ; Consulter ses avis (étendu par Signaler un avis) ; Gérer son profil ; Retirer ses gains	RF08 à RF13, RF20, RF22, RF23
Administrateur	Consulter le tableau de bord (étendu par Consulter le nombre d'utilisateurs et de prestataires, Consulter les statistiques des réservations, Consulter les revenus de la plateforme et Modérer les avis signalés) ; Gérer la validation des comptes prestataires (étendu par Consulter les justificatifs professionnels, puis Accepter ou refuser la demande d'inscription) ; Gérer les litiges et les sanctions (étendu par Consulter le détail d'un litige, Voir l'historique des sanctions et Trancher le litige)	RF14 à RF17, RF19, RF20
Service de notification (secondaire)	Envoyer une notification (inclus dans les cas d'utilisation concernés)	RF18
Prestataire de paiement DexPay (secondaire)	Participe à Payer l'acompte (encaissement) et à Retirer ses gains (versement)	RF21, RF23

7.2 Descriptions textuelles des cas d'utilisation

Les descriptions suivantes détaillent le déroulement des cas d'utilisation principaux : scénario nominal, scénarios alternatifs (erreurs et variantes), préconditions et postconditions.

7.2.1 Cas commun : S'authentifier

Tableau 8 – Description textuelle – S'authentifier

Élément	Description
Nom du CU	S'authentifier
Acteurs	Utilisateur (Client, Prestataire, Administrateur)
Objectif	Permettre à un utilisateur enregistré d'accéder de manière sécurisée aux fonctionnalités de la plateforme Sopi.
Préconditions	L'utilisateur possède un compte valide et actif (ni suspendu, ni banni).
Scénario nominal	1. L'acteur clique sur « Se connecter ». 2. Le système affiche le formulaire (e-mail, mot de passe).

Élément	Description
	3. L'acteur saisit et valide ses identifiants. 4. Le système vérifie les informations (mot de passe haché Bcrypt). 5. Le système redirige l'acteur vers la ressource demandée ou son tableau de bord.
Scénarios alternatifs	4a. Identifiants incorrects : le système affiche « Identifiant ou mot de passe incorrect » et réaffiche le formulaire. 4b. Cinq échecs consécutifs : le système bloque temporairement le compte et propose la procédure « Mot de passe oublié ». 4c. Compte suspendu ou banni : le système refuse l'accès et affiche le motif de la sanction.
Postconditions	L'utilisateur est reconnu par le système et accède aux fonctionnalités autorisées pour son rôle.

7.2.2 Cas du Client

Tableau 9 – Description textuelle – Rechercher un prestataire

Élément	Description
Nom du CU	Rechercher un prestataire
Acteurs	Client
Relations	Inclut « S'authentifier » ; étendu par « Filtrer les résultats » et « Consulter le profil d'un prestataire »
Objectif	Trouver un prestataire de proximité (mécanique, coiffure, pressing) dans le département de Keur Massar.
Préconditions	Le client est authentifié.
Scénario nominal	1. Le client accède à la barre de recherche. 2. Il saisit un domaine de métier ou un mot-clé. 3. Le système interroge la base de données et affiche les prestataires validés correspondants.
Scénarios alternatifs	3a. Aucun résultat : le système affiche « Aucun prestataire disponible dans cette catégorie ». 3b. Filtrer les résultats (extension) : le client filtre par quartier ou par disponibilité ; le système actualise la liste. 3c. Consulter le profil d'un prestataire (extension) : le client sélectionne un résultat ; le système affiche le catalogue, les tarifs, les photos et les évaluations.
Postconditions	Le client dispose de la liste des prestataires correspondants et peut consulter leurs profils.

Tableau 10 – Description textuelle – Réserver un créneau

Élément	Description
Nom du CU	Réserver un créneau
Acteurs	Client (principal) ; Service de notification (secondaire)
Relations	Inclut « S'authentifier », « Consulter l'agenda » et « Envoyer une notification »
Objectif	Permettre au client de demander un créneau de l'agenda d'un prestataire, en attendant la réponse de celui-ci.
Préconditions	

Élément	Description
	Le client est authentifié et a choisi une prestation sur le profil d'un prestataire. L'agenda est aussi consultable seul (« Consulter l'agenda »).
Scénario nominal	1. Le système affiche l'agenda du prestataire (« Consulter l'agenda », inclus). 2. Le client sélectionne un créneau libre ; le système le verrouille temporairement. 3. Le client clique sur « Confirmer la réservation ». 4. Le système vérifie que le verrou est toujours valide et enregistre la réservation avec le statut « En attente ». 5. Le système envoie une notification au prestataire (« Envoyer une notification », inclus), sans révéler l'identité complète du client. 6. Le système affiche au client le récapitulatif de la réservation et l'informe qu'il pourra payer l'acompte une fois sa demande acceptée.
Scénarios alternatifs	2a. Créneau déjà pris ou verrouillé par un autre client : le système refuse la sélection, actualise l'agenda et affiche un message d'erreur. 3a. Le client abandonne la réservation : le verrou temporaire expire de lui-même et le créneau redevient libre. 3b. Le client confirme après l'expiration du verrou : le système détecte l'expiration, libère le créneau, affiche le message « Délai dépassé » et invite le client à sélectionner de nouveau un créneau. 6a. Le prestataire ne répond pas dans les 12 heures : le système passe la réservation à « Annulée », libère le créneau et en informe le client.
Postconditions	La réservation « En attente » est enregistrée ; le créneau est bloqué ; elle apparaît sur les tableaux de bord du client et du prestataire.

Tableau 11 – Description textuelle – Payer l'acompte

Élément	Description
Nom du CU	Payer l'acompte
Acteurs	Client (principal) ; Prestataire de paiement (secondaire) ; Service de notification (secondaire)
Relations	Étend « Voir le détail d'une réservation » ; inclut « Envoyer une notification »
Objectif	Permettre au client de confirmer sa réservation en versant en ligne un acompte de 30 % du tarif de la prestation, après acceptation de sa demande par le prestataire (RF21).
Préconditions	1. Le client est authentifié. 2. La réservation a le statut « Acceptée, en attente de paiement » et le délai de paiement de 2 heures n'est pas écoulé.
Scénario nominal	1. Le client ouvre le détail de sa réservation acceptée et choisit « Payer l'acompte ». 2. Le système calcule l'acompte (30 % du tarif) et crée une session de paiement auprès du prestataire de paiement. 3. Le client choisit son opérateur (Wave ou Orange Money) sur la page de paiement sécurisée et valide l'opération sur son téléphone. 4. Le prestataire de paiement confirme le paiement au système. 5. Le système enregistre le paiement, passe la réservation à « Confirmée » et génère le code de réservation. 6. Le système notifie le prestataire que la réservation est confirmée et payée, et envoie au client sa confirmation avec le code de réservation (« Envoyer une notification », inclus).
Scénarios alternatifs	3a. Paiement refusé, échoué ou abandonné : le système enregistre le paiement comme « Échoué » et la réservation reste « Acceptée, en attente de paiement » ; le client peut réessayer tant que le délai de 2 heures n'est pas écoulé.

Élément	Description
	1a. Délai de paiement dépassé : le système a passé la réservation à « Annulée » et libéré le créneau ; l'action « Payer l'acompte » n'est plus proposée et le client est informé.
Postconditions	La réservation est « Confirmée », le paiement est « Réussi » et le client dispose du code de réservation à présenter le jour du rendez-vous. Le prestataire a été informé.

7.2.3 Cas du Prestataire

Tableau 12 – Description textuelle – Soumettre un dossier d'inscription

Élément	Description
Nom du CU	Soumettre un dossier d'inscription
Acteurs	Prestataire
Relations	Inclut « Joindre les justificatifs » ; étendu par « Suivre l'état de sa validation » ; n'inclut pas « S'authentifier »
Objectif	Permettre à un professionnel de soumettre ses informations et justificatifs afin d'ouvrir son profil sur la plateforme (RF08).
Préconditions	Le professionnel accède au formulaire d'inscription prestataire (aucun compte préalable n'est requis).
Scénario nominal	1. Le prestataire renseigne ses informations (nom de la structure, métier, adresse à Keur Massar) et choisit ses identifiants de connexion. 2. Il joint ses justificatifs : carte d'identité, diplôme ou attestation (« Joindre les justificatifs », inclus). 3. Il valide son dossier. 4. Le système crée le compte avec le statut « En attente de validation » et transmet le dossier à l'administrateur. 5. Le système affiche un écran indiquant que le dossier est en cours d'examen et envoie au prestataire, par e-mail, un lien de suivi personnel. Le prestataire n'a pas accès à la plateforme tant que son compte n'est pas « Actif ».
Scénarios alternatifs	2a. Document manquant ou format invalide : le système bloque la soumission et affiche « Veuillez joindre un fichier PDF ou image valide ». 5a. Suivre l'état de sa validation (extension) : le prestataire ouvre son lien de suivi, sans authentification, et consulte l'état de son dossier (En attente, Actif, Refusé) ainsi que le motif en cas de refus. 6a. Décision de l'administrateur : le prestataire reçoit par e-mail une notification indiquant l'acceptation (compte « Actif ») ou le refus avec le motif (compte « Refusé »). Après acceptation, il peut s'authentifier.
Postconditions	Le dossier est soumis et le compte est « En attente de validation ». Le prestataire ne peut être réservé qu'une fois son compte « Actif ».

Tableau 13 – Description textuelle – Configurer son agenda

Élément	Description
Nom du CU	Configurer son agenda
Acteurs	Prestataire ; Service de notification (secondaire, via l'extension)
Relations	Inclut « S'authentifier » ; étendu par « Déclarer une indisponibilité » (qui inclut « Envoyer une notification »)
Objectif	

Élément	Description
	Permettre au prestataire de définir ses plages horaires de travail (RF09) et de déclarer ses indisponibilités (RF12).
Préconditions	Le prestataire est authentifié et son compte est « Actif ».
Scénario nominal	1. Le prestataire ouvre « Configurer mon agenda ». 2. Le système affiche les plages déjà configurées (formulaire vide à la première utilisation). 3. Le prestataire définit ou modifie ses horaires et ses créneaux, puis valide.
Scénarios alternatifs	3a. Déclarer une indisponibilité (extension) : le prestataire sélectionne une plage (congé, urgence) ; le système bloque immédiatement les créneaux libres de la plage et présente au prestataire les réservations déjà enregistrées sur cette plage. Après sa confirmation, le système passe ces réservations à « Annulée », enregistre le remboursement intégral des acomptes déjà payés (paiement « Remboursé ») et notifie les clients concernés (« Envoyer une notification », inclus). 3b. Le prestataire renonce à annuler les réservations existantes : seuls les créneaux libres sont bloqués, les réservations conservées restent inchangées.
Postconditions	L'agenda est mis à jour en temps réel et appliqué à l'interface de réservation des clients. Les acomptes des réservations annulées sont remboursés.

Tableau 14 – Description textuelle – Gérer ses réservations

Élément	Description
Nom du CU	Gérer ses réservations
Acteurs	Prestataire ; Service de notification (secondaire)
Relations	Inclut « S'authentifier » ; étendu par « Traiter les demandes » et « Mettre à jour l'état de la prestation », qui incluent chacun « Envoyer une notification »
Objectif	Permettre au prestataire de traiter les demandes (RF10) et de suivre ses rendez-vous jusqu'à la fin de la prestation (RF11), qui déclenche le crédit de ses gains (RF22).
Préconditions	Le prestataire est authentifié et possède des réservations.
Scénario nominal	1. Le prestataire consulte son planning et la liste de ses réservations sur son tableau de bord. 2. Le système affiche les demandes « En attente », les rendez-vous à venir et les indicateurs d'activité.
Scénarios alternatifs	2a. Traiter les demandes (extension) — acceptation : dans un délai de 12 heures, le prestataire accepte une demande ; le système passe la réservation à « Acceptée, en attente de paiement », maintient le créneau bloqué et notifie le client, qui dispose de 2 heures pour payer l'acompte. 2b. Traiter les demandes (extension) — refus : le prestataire refuse en saisissant un motif ; le système passe la réservation à « Refusée », libère le créneau et notifie le client avec le motif. 2c. Demande sans réponse après 12 heures : le système passe la réservation à « Annulée », libère le créneau et en informe le client. 3a. Mettre à jour l'état de la prestation (extension) : le jour du rendez-vous, le prestataire saisit le code de réservation présenté par le client et passe la réservation « Confirmée » à « En cours », puis à « Terminée » ; le système enregistre chaque changement et notifie le client. 3b. Code de réservation invalide : le système refuse le changement d'état et réaffiche la saisie.

Élément	Description
	3c. Passage à « Terminée » : le système répartit l'acompte, retient la commission de la plateforme (10 % du tarif par défaut) et crédite le reste au portefeuille du prestataire.
Postconditions	Le cycle de vie de la réservation est à jour et visible sur le tableau de bord du client. Les gains d'une prestation terminée sont crédités au portefeuille du prestataire.

Tableau 15 – Description textuelle – Retirer ses gains

Élément	Description
Nom du CU	Retirer ses gains
Acteurs	Prestataire (principal) ; Prestataire de paiement (secondaire)
Relations	Inclut « S'authentifier »
Objectif	Permettre au prestataire de consulter son portefeuille et de retirer ses gains vers son compte mobile money (RF23).
Préconditions	1. Le prestataire est authentifié et son compte est « Actif ». 2. Son portefeuille présente un solde disponible.
Scénario nominal	1. Le prestataire ouvre « Mon portefeuille ». 2. Le système affiche le solde disponible, l'historique des gains (acompte, commission, part créditée pour chaque prestation) et l'historique des retraits. 3. Le prestataire choisit « Retirer », saisit le montant et son numéro mobile money (Wave ou Orange Money), puis confirme. 4. Le système vérifie le solde, débite le portefeuille et demande le virement au prestataire de paiement. 5. Le prestataire de paiement confirme le virement au système. 6. Le système enregistre le retrait avec le statut « Réussi » et met à jour le solde et l'historique.
Scénarios alternatifs	3a. Montant supérieur au solde disponible : le système refuse la demande et affiche le solde. 3b. Montant hors des limites fixées par l'opérateur : le système refuse la demande et affiche les limites. 5a. Virement échoué : le système enregistre le retrait avec le statut « Échoué », remet le montant dans le solde disponible et informe le prestataire.
Postconditions	Le portefeuille est à jour et chaque retrait est tracé dans l'historique.

7.2.4 Cas de l'Administrateur

Tableau 16 – Description textuelle – Gérer la validation des comptes prestataires

Élément	Description
Nom du CU	Gérer la validation des comptes prestataires
Acteurs	Administrateur (principal) ; Service de notification (secondaire)
Relations	Inclut « S'authentifier » ; étendu par « Consulter les justificatifs professionnels », lui-même étendu par « Accepter ou refuser la demande d'inscription », qui inclut « Envoyer une notification »
Objectif	Permettre à l'administrateur de contrôler l'accès des nouveaux prestataires à la plateforme (RF15).
Préconditions	

Élément	Description
	1. L'administrateur est authentifié. 2. Au moins un dossier a le statut « En attente de validation ».
Scénario nominal	1. L'administrateur accède à la liste des demandes d'inscription. 2. Il sélectionne le dossier d'un prestataire. 3. Il consulte les justificatifs professionnels (extension). 4. Il accepte la demande (extension « Accepter ou refuser la demande d'inscription »). 5. Le système passe le compte à « Actif ». 6. Le système notifie le prestataire par e-mail de la confirmation (« Envoyer une notification », inclus).
Scénarios alternatifs	3a. Justificatifs illisibles ou incomplets : l'administrateur laisse le dossier « En attente » et le traitera plus tard. 4a. Justificatifs non conformes : l'administrateur refuse la demande et saisit le motif ; le système passe le compte à « Refusé » et notifie le prestataire par e-mail avec le motif (« Envoyer une notification », inclus).
Postconditions	Le compte du prestataire est « Actif » (il peut s'authentifier) ou « Refusé » (accès non accordé). La décision est tracée.

Tableau 17 – Description textuelle – Gérer les litiges et les sanctions

Élément	Description
Nom du CU	Gérer les litiges et les sanctions
Acteurs	Administrateur (principal) ; Service de notification (secondaire)
Relations	Inclut « S'authentifier » ; étendu par « Consulter le détail d'un litige » et « Voir l'historique des sanctions » ; « Consulter le détail d'un litige » est étendu par « Trancher le litige », qui inclut « Envoyer une notification »
Objectif	Permettre à l'administrateur d'arbitrer un litige entre un client et un prestataire (RF17) et de gérer les sanctions, avec conservation de leur historique pour suivre les récidives (RF19).
Préconditions	1. L'administrateur est authentifié. 2. Au moins un litige a le statut « Ouvert ».
Scénario nominal	1. L'administrateur accède à la liste des litiges. 2. Il sélectionne un litige et consulte son détail : description du client, preuves, réponse du prestataire (extension). 3. Il consulte l'historique des sanctions de l'utilisateur concerné (extension). 4. Il tranche le litige (extension) et choisit la décision : Avertissement, Suspension temporaire ou Bannissement définitif. 5. Le système enregistre la décision et la sanction dans l'historique, applique ses effets au compte et passe le litige à « Clôturé ». 6. Le système notifie les parties concernées (« Envoyer une notification », inclus).
Scénarios alternatifs	4a. Signalement non fondé : l'administrateur choisit « Rejet du signalement » ; aucune sanction n'est enregistrée ; le système clôture le litige et notifie les parties. 4b. Litige déjà clôturé : le système refuse une nouvelle décision.
Postconditions	Le litige est « Clôturé ». La sanction éventuelle est enregistrée dans l'historique et appliquée au compte : un compte suspendu ou banni ne peut plus s'authentifier.

Tableau 18 – Description textuelle – Consulter le tableau de bord

Élément	Description
Nom du CU	Consulter le tableau de bord
Acteurs	Administrateur
Relations	Inclut « S'authentifier » ; étendu par « Consulter le nombre d'utilisateurs et de prestataires », « Consulter les statistiques des réservations », « Consulter les revenus de la plateforme » et « Modérer les avis signalés »
Objectif	Offrir à l'administrateur une vue d'ensemble de l'activité de la plateforme dans le département de Keur Massar et un point d'entrée vers les actions de supervision (RF14, RF16).
Préconditions	L'administrateur est authentifié.
Scénario nominal	1. L'administrateur ouvre son tableau de bord. 2. Le système affiche les indicateurs de synthèse à jour : nombre d'utilisateurs et de prestataires, réservations par statut. 3. Le système affiche des bannières d'alerte signalant les éléments en attente de traitement : dossiers en attente de validation, litiges ouverts et avis signalés.
Scénarios alternatifs	2a. Consulter le nombre d'utilisateurs et de prestataires (extension) : le système détaille les effectifs par rôle. 2b. Consulter les statistiques des réservations (extension) : le système affiche le nombre de réservations par statut, par période et par domaine (mécanique, coiffure, pressing). 2c. Consulter les revenus de la plateforme (extension) : le système affiche le montant des acomptes encaissés, la commission retenue par la plateforme et les montants retirés par les prestataires, par période. 2d. Modérer les avis signalés (extension) : l'administrateur consulte un avis signalé par un prestataire, puis le supprime ou le conserve ; le système applique la décision et met à jour l'avis. 3a. Depuis une bannière d'alerte, l'administrateur ouvre « Gérer la validation des comptes prestataires » ou « Gérer les litiges et les sanctions ». 3b. Aucun élément en attente : aucune bannière d'alerte n'est affichée.
Postconditions	L'administrateur dispose de données à jour. Les avis non conformes sont modérés.

7.3 Diagrammes de séquence d'analyse

Les diagrammes de séquence illustrent comment des scénarios spécifiques se déroulent au fil du temps. Les trois scénarios modélisés ont été sélectionnés pour leur criticité (gestion des accès concurrents sur les créneaux, paiement de l'acompte), leur complexité (mise à jour des agendas, échanges avec le prestataire de paiement et notifications) et leur représentativité du fonctionnement global de Sopi à Keur Massar. Ils couvrent les flux nominaux essentiels ainsi que les scénarios d'erreur. Chaque diagramme met en évidence les participants techniques fondamentaux du scénario, à savoir la base de données (:BDD), le service de notification (:NotificationService) et le prestataire de paiement (:DexPay), autour du composant applicatif principal :Systeme.

7.3.1 Client : Réserver un créneau

Le client consulte l'agenda d'un prestataire de Keur Massar via le :Systeme, qui lit les créneaux dans la :BDD. Lorsque le client sélectionne un créneau, le :Systeme demande à la :BDD de le verrouiller de façon atomique (« verrouiller si libre »). Cette opération garantit qu'un seul client obtient le créneau, même en cas d'accès concurrents. Si le verrou est obtenu, le :Systeme demande la confirmation du

client. Lorsque celui-ci confirme dans le délai du verrou, la :BDD enregistre la réservation avec le statut « En attente », puis le :Systeme sollicite le :NotificationService, qui alerte le prestataire de la nouvelle demande ; le récapitulatif est ensuite affiché au client. Aucun paiement n'est demandé à ce stade. Si le verrou a expiré avant la confirmation, le créneau est libéré et un message d'erreur est affiché. Si le créneau est déjà verrouillé ou réservé, le :Systeme relit l'agenda et affiche l'erreur accompagnée de l'agenda actualisé.

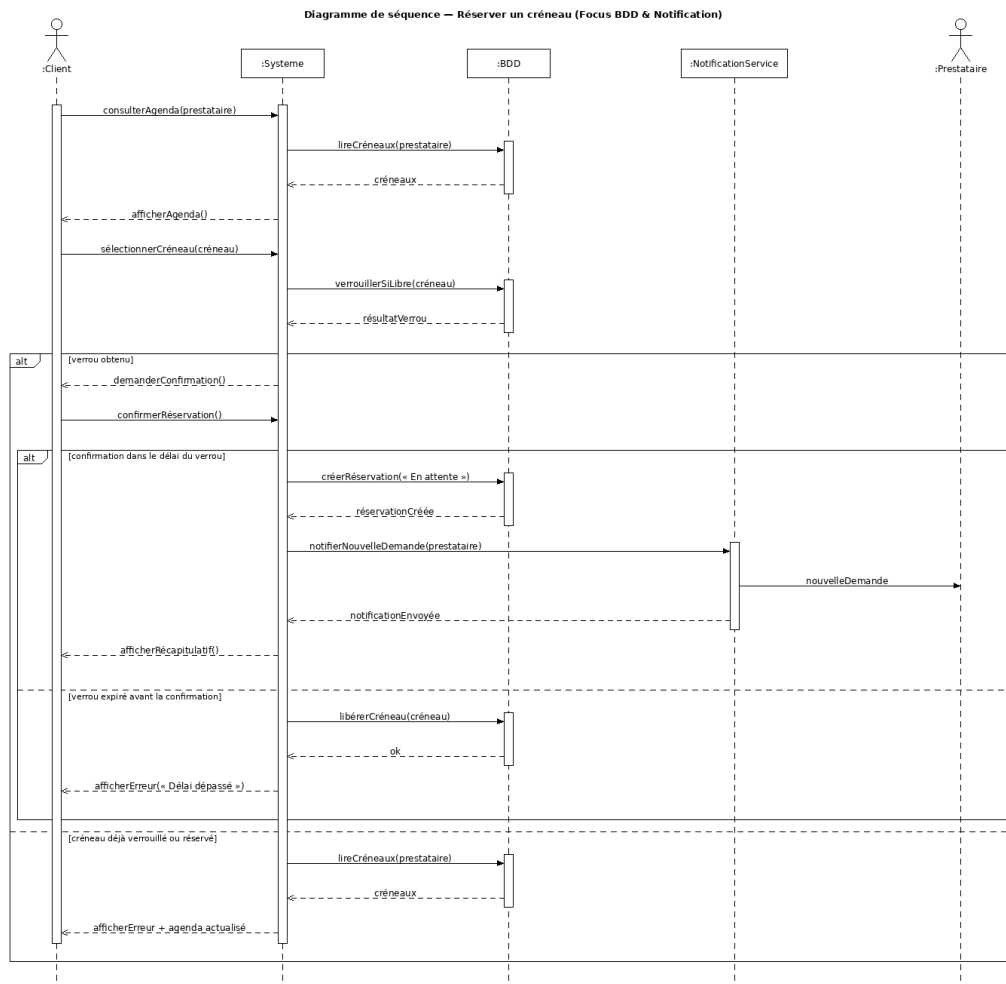


Figure 5 – Diagramme de séquence « Réserver un créneau » (focus BDD et notification)

7.3.2 Prestataire : Traitement d'une demande

Le prestataire consulte, via le :Systeme, la liste des demandes de réservation « En attente » lues dans la :BDD. Il examine ensuite chaque demande et décide de l'accepter ou de la refuser. S'il accepte, le :Systeme fait passer la réservation au statut « Acceptée » (en attente de paiement) dans la :BDD et le créneau reste bloqué sur l'agenda. Le :NotificationService informe alors le client que sa demande est acceptée et qu'il doit payer l'acompte pour la confirmer ; la réservation ne devient « Confirmée » qu'après ce paiement (séquence suivante). S'il refuse, il saisit un motif : la réservation passe à « Refusée » et le créneau est libéré pour d'autres clients. Dans les deux cas, le :Systeme sollicite le :NotificationService, qui informe automatiquement le client de la décision (acceptation avec invitation à payer l'acompte, ou refus accompagné du motif).

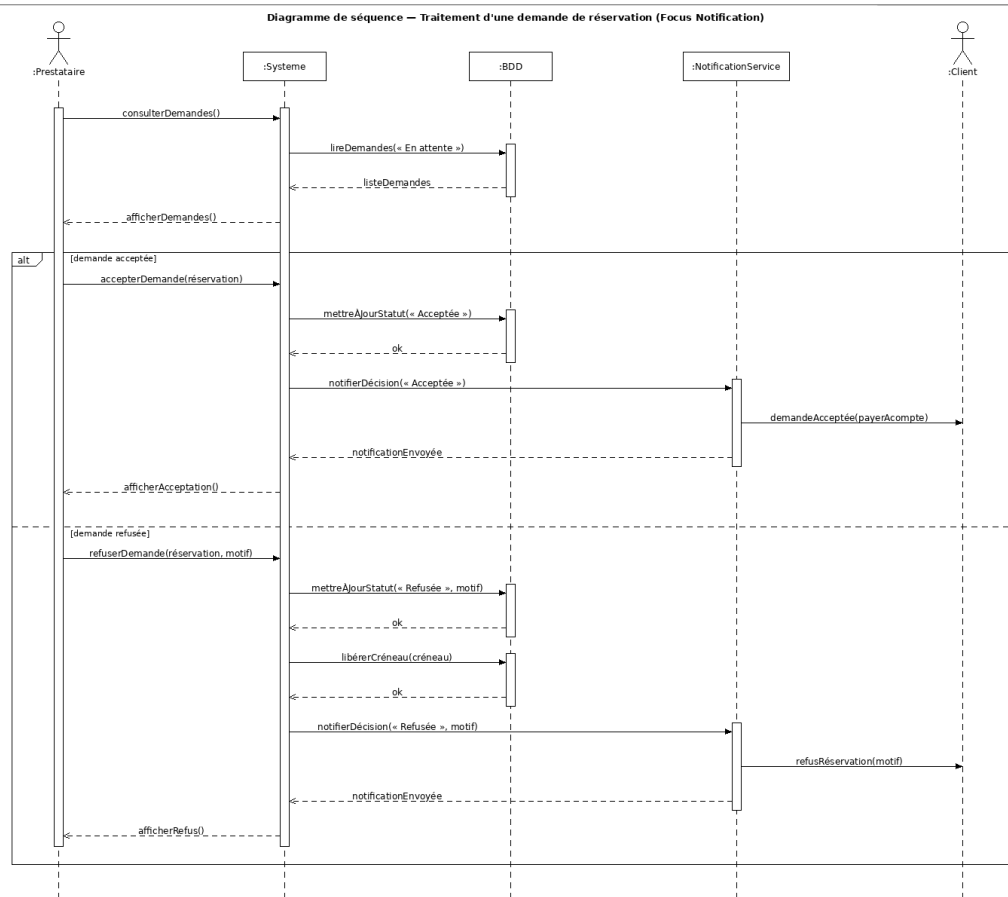


Figure 6 – Diagramme de séquence « Traitement d'une demande de réservation » (focus notification)

7.3.3 Client : Payer l'acompte

Le client dont la demande est « Acceptée » demande à payer l'acompte. Le :Systeme lit la réservation et son tarif dans la :BDD, puis demande au prestataire de paiement :DexPay de créer une session de paiement d'un montant égal à 30 % du tarif. :DexPay renvoie l'adresse de la page de paiement, vers laquelle le client est redirigé ; il y choisit Wave ou Orange Money et valide l'opération sur son téléphone. Le :Systeme ne traite jamais de données bancaires. :DexPay informe ensuite le :Systeme du résultat par un webhook signé. Si le paiement est réussi, le :Systeme enregistre le paiement (« REUSSI »), fait passer la réservation à « Confirmée » et génère le code de réservation. Le :NotificationService confirme alors le paiement au client en lui transmettant ce code, et informe le prestataire que la réservation est confirmée ; le code est aussi affiché au client dans l'application. À l'heure du rendez-vous, le prestataire saisit ce code pour démarrer la prestation, ce qui lui permet d'identifier le client. Si le paiement échoue, il est enregistré avec le statut « ECHOUÉ » et le client peut réessayer. Sans paiement dans le délai de 2 heures, la réservation est annulée et le créneau est libéré.

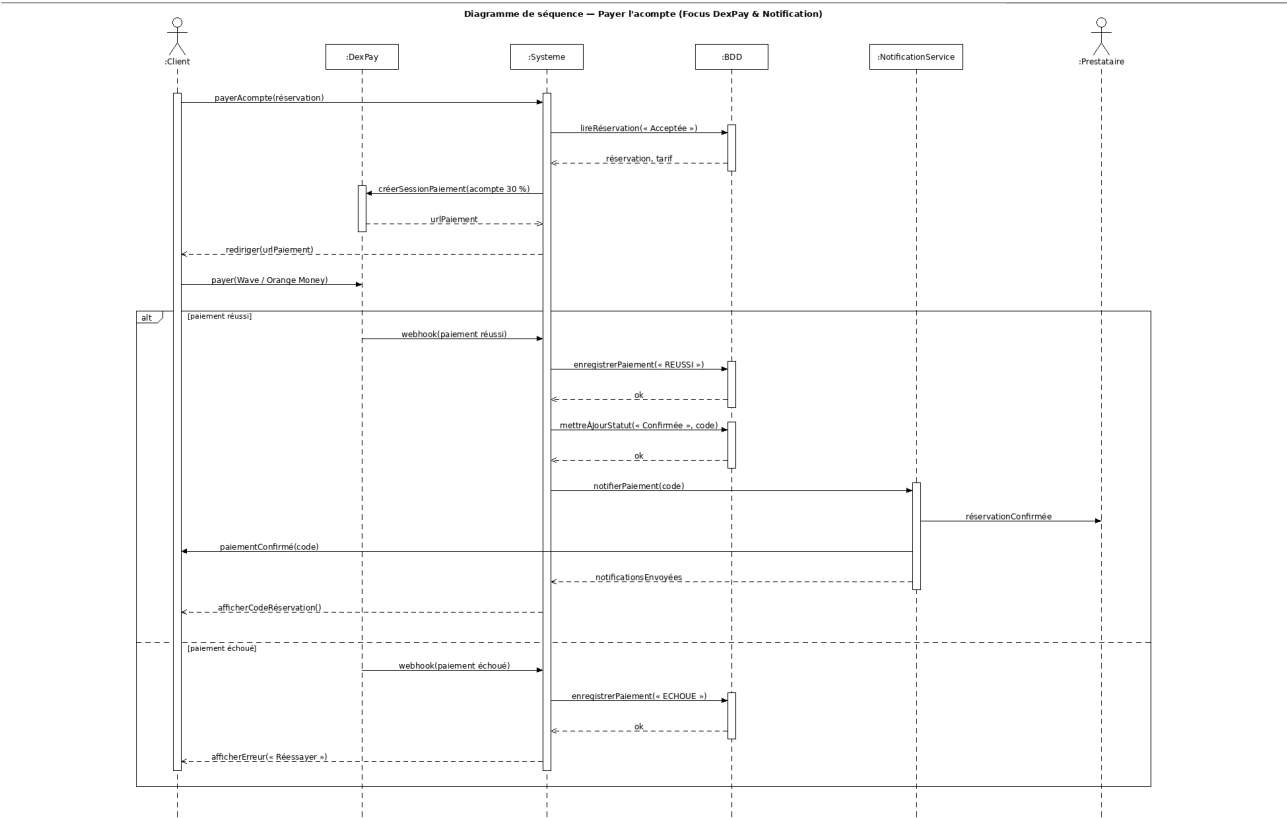


Figure 7 – Diagramme de séquence « Payer l'acompte » (focus DexPay et notification)

8. Modèle de données

8.1 Diagramme de classes d'analyse

Le diagramme de classes d'analyse modélise la structure statique du système : il identifie les concepts métier fondamentaux du domaine des services de proximité à Keur Massar et les relations qui les unissent. Il se concentre sur le « quoi » et non sur le « comment » : il présente les concepts métier, leurs informations essentielles et leurs associations, sans méthodes ni détails d'implémentation. Chaque classe et chaque attribut correspond à un élément utilisé dans les cas d'utilisation ou dans les diagrammes de séquence. Les classes sont regroupées en quatre packages logiques :

- **Gestion des utilisateurs et des portefeuilles** : la classe abstraite Utilisateur (identité, coordonnées, statut du compte) et ses trois spécialisations Client, Prestataire et Administrateur, ainsi que les Justificatifs qui constituent le dossier d'inscription d'un prestataire. Le statut du compte (En attente de validation, Actif, Refusé, Suspendu, Banni) pilote l'accès à la plateforme. Chaque prestataire possède un Portefeuille, qui accumule ses gains, et peut effectuer des Retraits depuis ce portefeuille.
- **Catalogue des prestations** : la classe Service décrit une prestation générique (vidange, lavage, coupe) ; la classe PrestationPrestataire représente ce service tel qu'un prestataire donné le propose, avec sa durée estimée et son tarif.
- **Agenda, réservations et paiements** : le Créneau est une plage horaire d'un prestataire, avec son statut et l'heure d'expiration de son verrou temporaire ; la Réservation est le rendez-vous pris par un client, dont le statut suit le cycle En attente, Acceptée (en attente de paiement), Confirmée, En cours, Terminée, Refusée ou Annulée, et qui porte le code de réservation remis au client après le paiement ; le Paiement représente l'acompte réglé par le client via DexPay, avec son statut (en attente, réussi, échoué ou remboursé) et la commission retenue par la plateforme.
- **Support, modération et régulation** : l'Avis, le Litige, la Sanction et la Notification, qui instaurent et maintiennent la confiance entre les acteurs et conservent l'historique des sanctions.

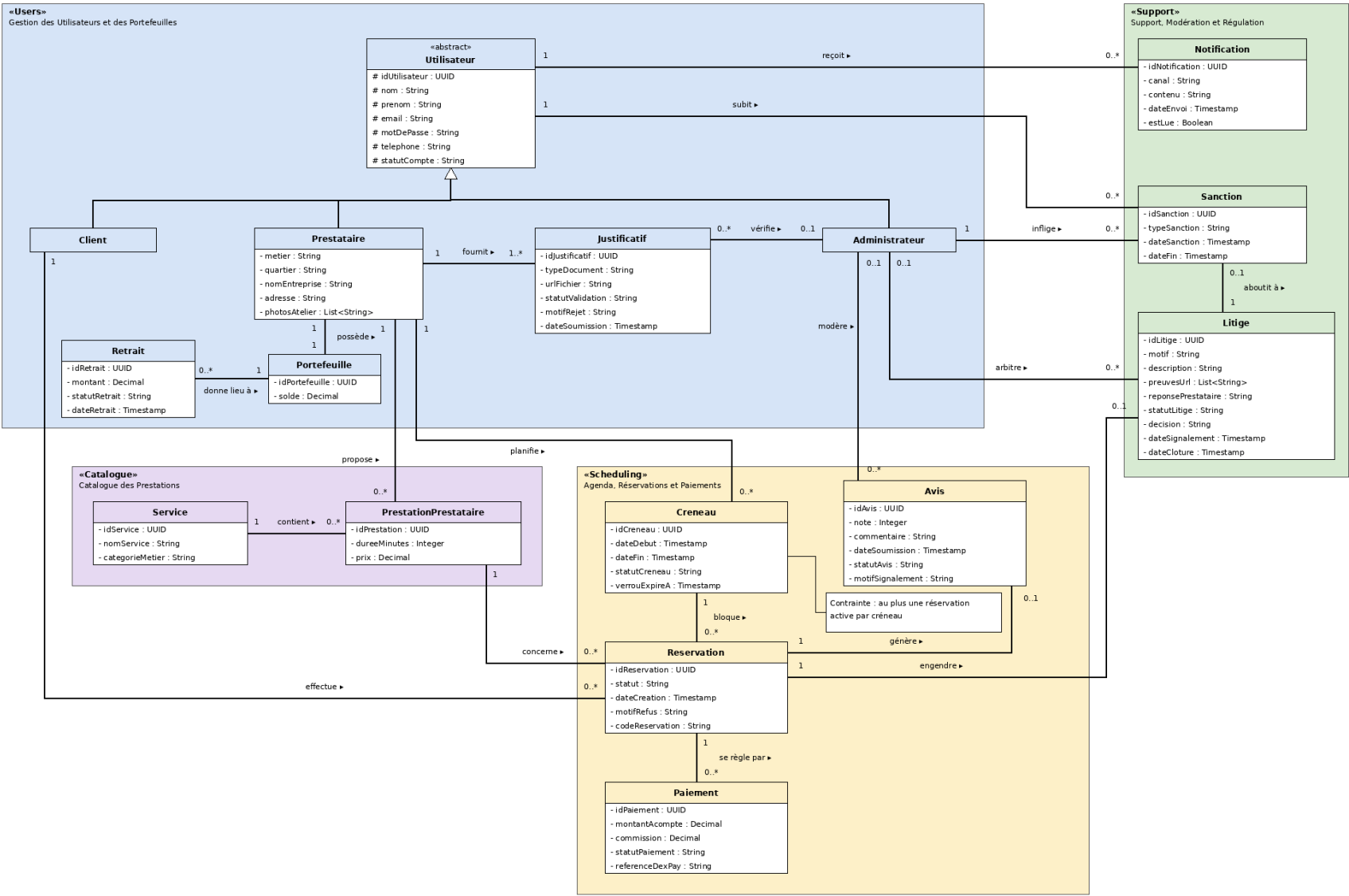


Figure 8 – Diagramme de classes d'analyse de la plateforme Sopi

Tableau 19 – Description des classes d'analyse du système Sopi

Classe	Responsabilité
Utilisateur (abstraite)	Regroupe les informations communes (identifiant, nom, prénom, e-mail, mot de passe, téléphone) et le statut du compte de tous les acteurs de la plateforme.
Client	Hérite d'Utilisateur. Représente l'utilisateur qui recherche des services de proximité, effectue des réservations et paie l'acompte.
Prestataire	Hérite d'Utilisateur. Représente le professionnel local (mécanicien, coiffeur, pressing) caractérisé par son entreprise, son adresse, son quartier d'activité à Keur Massar et les photos de son atelier.
Justificatif	Représente un document officiel (pièce d'identité, diplôme, attestation) fourni par le prestataire, vérifié par l'administrateur, avec son statut de validation et, le cas échéant, le motif de rejet.
Administrateur	Hérite d'Utilisateur. Représente le gestionnaire de la plateforme chargé de la modération, de la vérification des documents, du règlement des litiges et du suivi des revenus.
Portefeuille	Représente le solde des gains d'un prestataire : après chaque prestation terminée, la part de l'acompte revenant au prestataire (acompte moins commission) y est créditée.
Retrait	Enregistre une demande de retrait d'un prestataire depuis son portefeuille, avec son montant, son statut (en cours, réussi ou échoué) et sa date.
Service	Représente les catégories de services génériques rattachées à un métier (ex : vidange, coupe homme, nettoyage à sec).
PrestationPrestataire	Classe pivot reliant un Service à un Prestataire. Elle définit le prix spécifique et la durée estimée fixés par chaque artisan.
Créneau	Représente une plage horaire planifiée par un prestataire pour indiquer ses disponibilités. Il porte un statut et un verrou temporaire qui expire, utilisé pendant la réservation pour éviter les accès concurrents.
Réservation	Représente le rendez-vous entre un client et une prestation sur un créneau donné, et suit son état (en attente, acceptée, confirmée, en cours, terminée, refusée ou annulée). Elle porte le code de réservation généré après le paiement de l'acompte, que le prestataire saisit pour démarrer la prestation.
Paielement	Représente l'acompte (30 % du tarif) réglé par le client via DexPay, avec son montant, la commission retenue par la plateforme, sa référence DexPay et son statut (en attente, réussi, échoué ou remboursé).
Avis	Permet au client de noter et commenter une prestation une fois la réservation terminée. Il peut être signalé pour modération.
Litige	Enregistre la réclamation déclarée par un client suite à une réservation, avec sa description, ses preuves et la réponse du prestataire, en attente d'arbitrage par l'administrateur.
Sanction	Contient la décision disciplinaire (avertissement, suspension temporaire, bannissement définitif) infligée par l'administrateur à un utilisateur (client ou prestataire) suite à la résolution d'un litige.
Notification	Gère les messages d'alerte et de suivi envoyés aux utilisateurs pour les informer des événements de leur compte, avec leur état de lecture.

8.2 Diagramme de classes de conception

Le diagramme de classes de conception traduit le modèle de domaine en une structure logicielle concrète, prête pour l'implémentation. Il répond à la question : comment allons-nous construire le système ? Le passage du modèle d'analyse au modèle de conception se matérialise par un enrichissement significatif du diagramme :

- **Précision des attributs et encapsulation** : les types de données sont spécifiés avec des types techniques (UUID, DateTime, Decimal, Boolean) et la visibilité de chaque attribut est définie (majoritairement privée « - », ou protégée « # » pour la classe abstraite Utilisateur). Les méthodes publiques de chaque classe reprennent les cas d'utilisation et les diagrammes de séquence validés.
- **Introduction des énumérations** : les attributs représentant des statuts ou des types fixes (StatutReservation, StatutCreneau, StatutCompte, StatutPaiement, StatutRetrait, DecisionLitige) sont modélisés comme des énumérations plutôt que de simples chaînes de caractères.
- **Affinement des relations et éléments de support** : les relations sont détaillées avec des multiplicités précises et des clés étrangères (creneauld, prestationId et clientId dans Reservation, modereParId dans Avis, reservationId dans Paiement). L'interface **NotificationService** isole l'envoi des notifications (e-mail, push) du reste du système.
- **Intégration du paiement** : les classes Paiement, Portefeuille et Retrait portent la gestion financière. Un Paiement enregistre l'acompte de 30 % réglé par le client, la commission retenue par la plateforme (10 % du tarif, paramètre du système) et la référence de la transaction DexPay. Le Portefeuille cumule les gains du prestataire, crédités à la fin de chaque prestation, et chaque Retrait en trace le versement. L'interface **PaiementService** isole le prestataire de paiement DexPay (création de la session de paiement, vérification de la signature des webhooks, versement des gains), de la même façon que NotificationService.
- **Code de réservation** : la réservation reçoit un attribut codeReservation, généré lorsque le paiement est confirmé et que le prestataire saisit pour démarrer la prestation (demarrer(code)).

Tableau 20 – Énumérations du modèle de conception

Énumération	Valeurs
StatutReservation	EN_ATTENTE, ACCEPTEE (en attente de paiement), CONFIRMEE, EN_COURS, TERMINEE ; issues : REFUSEE, ANNULEE
StatutPaiement	EN_ATTENTE, REUSSI, ECHOU, REMBOURSE
StatutRetrait	EN_COURS, REUSSI, ECHOU
StatutCompte	En attente de validation, Actif, Refusé, Suspendu, Banni
StatutCreneau	statut du créneau (libre, verrouillé temporairement, réservé) associé à l'heure d'expiration du verrou
DecisionLitige	Rejet du signalement, Avertissement, Suspension temporaire, Bannissement définitif

Ce diagramme représente le plan directeur (blueprint) de l'application. Il sert de guide pour le développement en décrivant la structure des objets manipulés par le backend Laravel et stockés dans la base de données relationnelle. Il n'inclut pas toutes les classes purement techniques (contrôleurs, routes) pour des raisons de lisibilité. Chaque classe correspond à une table PostgreSQL et chaque association à une clé étrangère.

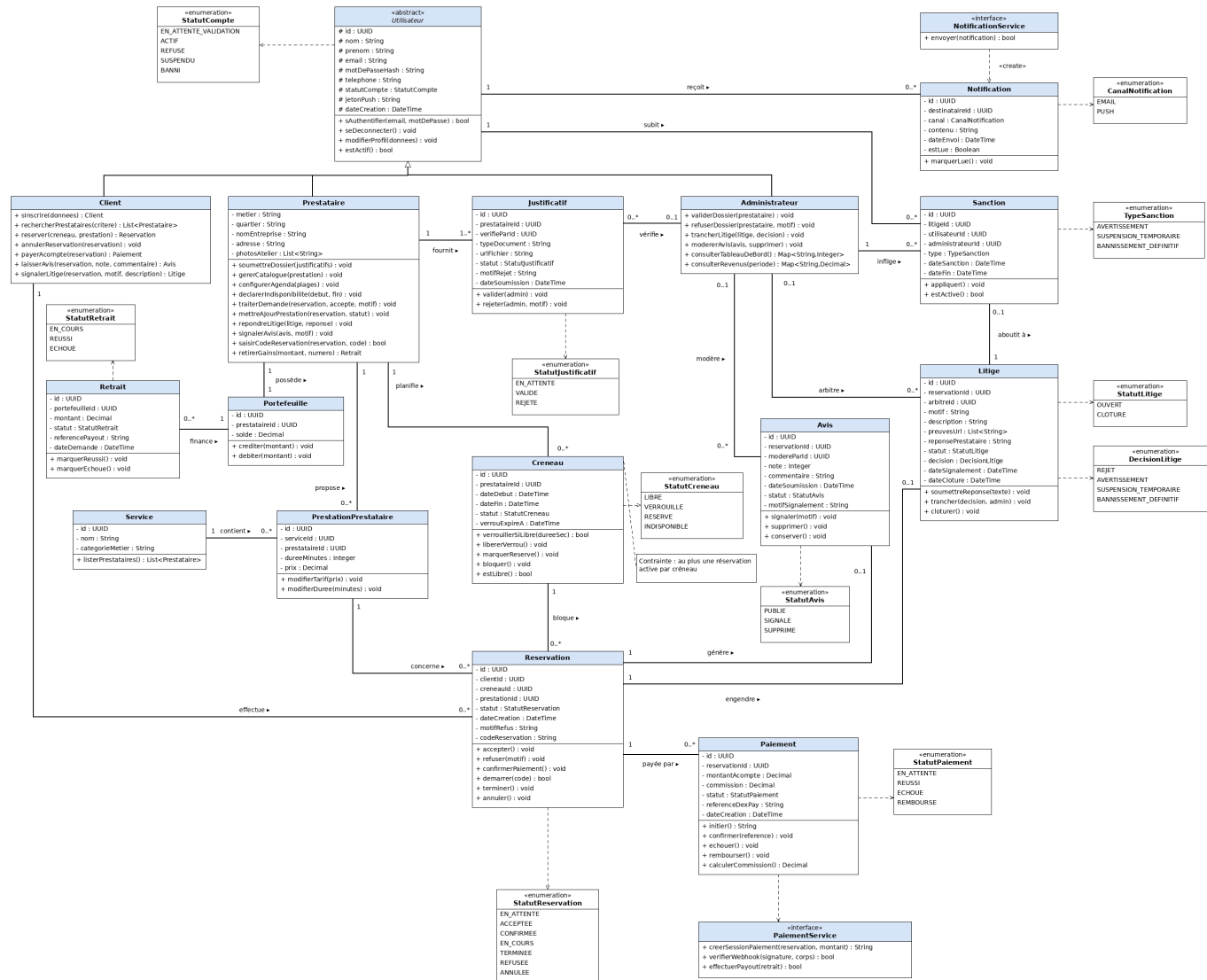


Figure 9 – Diagramme de classes de conception de la plateforme Sopi

9. Spécifications techniques

9.1 Technologies et outils

L'architecture retenue privilégie une approche full-stack moderne : un backend Laravel exposant une API REST sécurisée, deux interfaces consommant cette API (une application mobile Flutter pour les clients et les prestataires, et un back-office web React pour l'administrateur), une base de données relationnelle PostgreSQL, des services de notification (e-mail et push) et la passerelle de paiement mobile DexPay pour le règlement de l'acompte. Le backend, le back-office et la base de données sont hébergés sur un serveur géré par la plateforme Dokploy, et l'application mobile est compilée en fichier APK, téléchargeable et installable sur les téléphones Android.

Le choix des technologies s'est effectué selon plusieurs critères : la performance, la sécurité, la maintenabilité, l'adéquation avec les besoins des acteurs (gestion des accès concurrents sur les créneaux, sécurité des paiements) et les contraintes du contexte sénégalais : coût de développement et de maintenance, disponibilité de la documentation et d'une large communauté, usage prioritaire du téléphone mobile, paiement par Wave et Orange Money, et fonctionnement correct sur des connexions à faible débit. Ces contraintes ont orienté le choix vers des solutions open source et des technologies éprouvées.

9.1.1 Langages de programmation

Tableau 21 – Langages de programmation utilisés

Langage	Version	Rôle et justification
PHP	8.x	Langage natif de Laravel : toute la logique métier (réservations, créneaux, paiements, litiges) est écrite en PHP. Langage très répandu, documentation abondante, hébergement économique adapté au budget du projet.
Dart	3.x	Langage de Flutter : permet de développer l'application mobile des clients et des prestataires. Compilation en code natif pour Android et iOS ; une seule base de code pour plusieurs systèmes.
JavaScript	ES6+	Langage de React : utilisé pour le back-office web de l'administrateur (tableaux, statistiques, validation des dossiers). Très large écosystème de bibliothèques, exécuté directement dans le navigateur.
SQL	Standard	Langage universel des bases de données relationnelles ; adapté aux requêtes avec jointures (réservations, créneaux, prestataires, paiements) ; utilisé via l'ORM Eloquent sur PostgreSQL.
HTML / CSS	HTML5, CSS3	Standards du web : structure et mise en forme des pages du back-office, affichage correct sur les navigateurs récents.

9.1.2 Frameworks et services

Tableau 22 – Frameworks et services utilisés

Technologie	Version	Rôle dans Sopi
Laravel	11.x	Framework PHP open source (modèle MVC), cœur du backend : expose l'API REST, applique les règles métier (verrouillage des créneaux, cycle de vie des réservations, calcul de la commission, gestion du portefeuille, décisions de l'administrateur), dialogue avec DexPay et pilote l'envoi des notifications. Fournit nativement l'authentification, la validation des données, le hachage Bcrypt, la protection contre les attaques courantes (CSRF, injection SQL) et un système de notifications.
Flutter	3.x	Framework mobile multiplateforme de Google (langage Dart). Couche de présentation mobile des clients et des prestataires : recherche, agenda, réservation, paiement de l'acompte, suivi des réservations, gestion du catalogue et des demandes, portefeuille et retrait des gains. Consomme l'API REST de Laravel et reçoit les notifications push de Firebase. Compilée en APK Android.
React	18.x	Bibliothèque JavaScript (Meta) du back-office web de l'administrateur : validation des dossiers d'inscription, arbitrage des litiges et application des sanctions, modération des avis signalés, tableau de bord (indicateurs, revenus de la plateforme, alertes). Un écran large et des tableaux de données conviennent mieux à ce travail qu'une application mobile.
PostgreSQL	16.x	Système de gestion de bases de données relationnelles open source. Gère les transactions de manière rigoureuse, indispensable pour éviter qu'un même créneau soit réservé par deux clients et pour que les opérations financières restent cohérentes. Stocke utilisateurs, justificatifs, services, créneaux, réservations, paiements, portefeuilles, retraits, avis, litiges, sanctions et notifications, via l'ORM Eloquent (une classe = une table).
DexPay	mode test (sandbox)	Passerelle de paiement mobile (Wave, Orange Money) fonctionnant par « session de paiement » : le backend demande la création d'une session, le client est redirigé vers une page de paiement hébergée par DexPay et valide l'opération sur son téléphone. L'application ne manipule jamais de numéro de carte ni de code secret. Le résultat revient par un webhook signé (HMAC-SHA256) vérifié par le backend. Une API de versement (payout) permet de reverser les gains aux prestataires. Configurée avec des clés de test : transactions simulées, aucun argent réel. Isolée derrière l'interface PaiementService.
Laravel Mail (SMTP)	serveur SMTP de Gmail	Notifications par e-mail : résultat de l'examen d'un dossier de prestataire (avec lien de suivi), acceptation ou refus d'une demande, confirmation de paiement avec le code de réservation, décision de l'administrateur sur un litige.
Firebase Cloud Messaging	—	Notifications push gratuites, même lorsque l'application n'est pas ouverte : nouvelle demande ou réservation confirmée pour le prestataire ; acceptation, confirmation de paiement ou refus pour le client. Isolé, avec l'e-mail, derrière l'interface NotificationService.
Dokploy	—	Plateforme open source de déploiement installée sur un serveur : se connecte au dépôt GitHub, récupère le code, le construit, le met en ligne et gère bases de données, variables d'environnement, certificat HTTPS et mises à jour. Héberge le backend Laravel, le back-office React et la base PostgreSQL.

Tableau 23 – Synthèse des couches et de leurs interconnexions

Technologie	Couche	Rôle principal	Interconnexion
Flutter	Frontend mobile	Application des clients et des prestataires (APK Android)	Consomme l'API REST de Laravel (JSON via HTTPS) ; reçoit les notifications push
React	Frontend web	Back-office de l'administrateur	Consomme l'API REST de Laravel (JSON via HTTPS)
Laravel	Backend	API REST, règles métier, paiements, authentification	Expose les routes à Flutter et à React ; accède à PostgreSQL via Eloquent ; appelle DexPay
PostgreSQL	Données	Stockage relationnel et transactions	Utilisé par Laravel
DexPay	Paieement	Encaissement de l'acompte (Wave, Orange Money) et versement des gains	Appelé par Laravel ; renvoie le résultat par webhook signé
Laravel Mail	Notification	Envoi des e-mails via SMTP (Gmail)	Appelé par Laravel à chaque événement important
Firebase (FCM)	Notification	Envoi des notifications push	Appelé par Laravel, reçu par l'application mobile
Dokploy	Hébergement	Déploiement et hébergement de l'API, du back-office et de la base	Déploie le code depuis GitHub ; expose l'API en HTTPS

9.1.3 Outils de développement

Tableau 24 – Outils de développement utilisés

Catégorie	Outil	Usage dans le projet
IDE et éditeurs	Visual Studio Code	Éditeur principal pour le backend (PHP), l'application mobile (Dart) et le back-office (React). Extensions : PHP Intelephense, Laravel, Flutter et Dart ; débogage intégré, terminal intégré, complétion de code.
Développement mobile	Android Studio	Émulateur Android et SDK pour tester l'application Flutter sur différents appareils et pour produire l'APK.
Développement mobile	Flutter SDK	Outils de compilation et de débogage de l'application mobile ; commande de génération de l'APK de production.
Gestion des dépendances	Composer	Gestionnaire de paquets PHP : installation de Laravel et de ses bibliothèques.
Gestion des dépendances	Node.js	Environnement d'exécution JavaScript nécessaire au back-office React.
Gestion des dépendances	npm	Gestionnaire de paquets JavaScript : installation de React et lancement du back-office en développement.
Gestion de versions	Git	Système de contrôle de versions décentralisé : historique complet du projet avec ses commits.
Gestion de versions	GitHub	Hébergement distant du dépôt Git : sauvegarde du code et source des déploiements.

Catégorie	Outil	Usage dans le projet
Déploiement	Dokploy	Déploiement et hébergement du backend, du back-office et de la base de données sur le serveur, gestion du HTTPS et des variables d'environnement.
Modélisation UML	draw.io	Réalisation des diagrammes : cas d'utilisation, séquence, classes d'analyse et de conception.
Test et validation	Postman	Plateforme de test d'API REST : vérification des routes d'authentification, de réservation, de paiement, de validation des dossiers et de litiges.
Test et validation	DexPay (mode test)	Environnement de test de la passerelle de paiement : clés de test et paiements simulés pour valider le parcours de l'acompte sans argent réel.
Rédaction	Overleaf (LaTeX)	Éditeur LaTeX en ligne utilisé pour rédiger et mettre en forme le mémoire.

9.2 Génération de l'application Android (APK)

L'application mobile est livrée sous la forme d'un fichier APK, le format d'installation des applications Android. Il est produit par la commande `flutter build apk --release` du Flutter SDK, qui compile le code Dart en code natif et l'empaquette avec la configuration Firebase. Le fichier obtenu est mis à disposition en téléchargement : il suffit au client ou au prestataire de le télécharger sur son téléphone Android et de l'installer, après avoir autorisé l'installation d'applications provenant de sources externes. La publication sur une boutique d'applications (Google Play) n'est pas nécessaire pour la démonstration ; elle est envisagée comme perspective.

9.3 Architecture technique

L'architecture de Sopi repose sur un modèle en couches, qui sépare l'interface, les règles métier, le stockage des données et l'envoi des notifications. Elle fait aussi appel à un service externe de paiement (DexPay) et s'appuie sur un serveur d'hébergement géré par Dokploy. Cette séparation rend l'application modulaire : une couche peut évoluer (par exemple l'ajout d'un nouveau canal de notification ou d'un nouveau prestataire de paiement) sans remettre en cause les autres.

9.3.1 Vue d'ensemble : architecture en couches

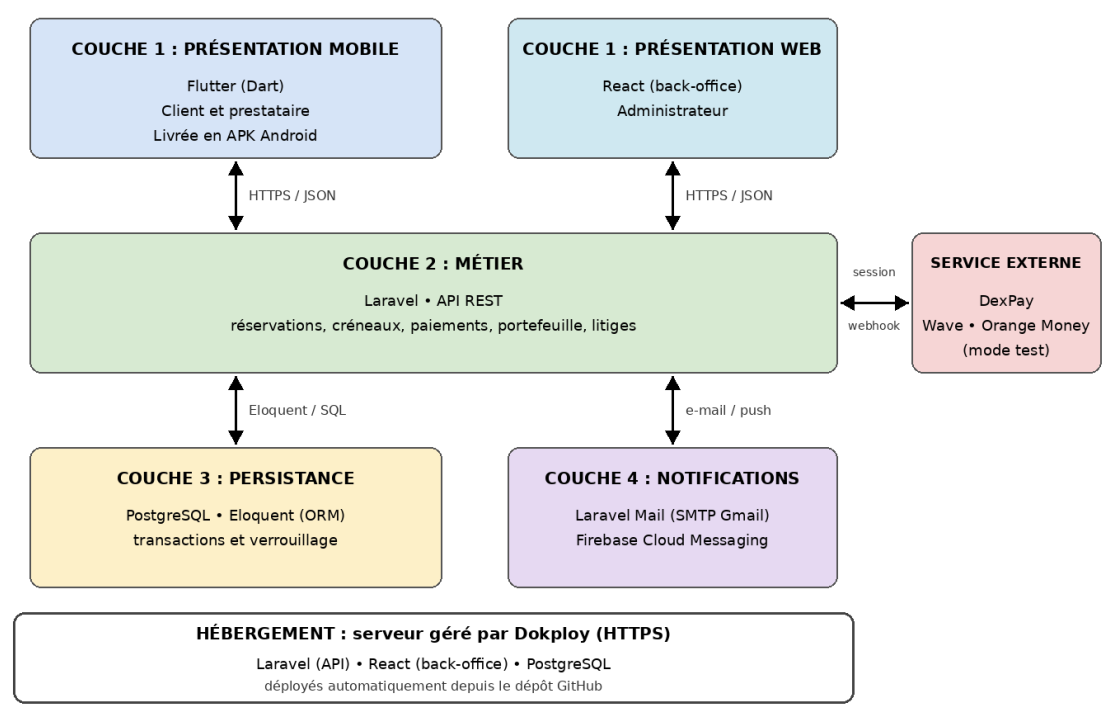


Figure 10 – Architecture en couches de Sopi

Le client et le prestataire utilisent l'application mobile Flutter sur leur téléphone Android, installée à partir du fichier APK ; l'administrateur utilise le back-office web React dans son navigateur. Chaque action passe par une requête HTTPS à l'API REST de Laravel, qui applique les règles métier, lit ou écrit dans PostgreSQL, puis déclenche si nécessaire l'envoi d'une notification par e-mail ou push. Pour le paiement de l'acompte, l'API dialogue avec le service externe DexPay : elle lui demande de créer une session de paiement, puis reçoit le résultat par un webhook. Le backend, le back-office et la base de données sont déployés sur un serveur géré par Dokploy.

9.3.2 Description des couches

Tableau 25 – Description des couches de l'architecture

Couche	Description
Couche 1 : Présentation (Flutter et React)	Deux interfaces. L'application mobile Flutter s'adresse au client (recherche, agenda, réservations, paiement de l'acompte, avis, litiges) et au prestataire (dossier, catalogue, agenda, demandes, saisie du code de réservation, portefeuille et retrait des gains) ; elle est pensée pour un usage quotidien sur téléphone. Le back-office web React s'adresse à l'administrateur (validation des dossiers, litiges et sanctions, modération, tableau de bord avec indicateurs, revenus et bannières d'alerte). Ni l'une ni l'autre ne contient de règle métier : elles affichent les données reçues et envoient les actions de l'utilisateur à l'API.

Couche	Description
Couche 2 : Métier (Laravel)	API REST dont les routes sont groupées par domaine métier (authentification, client, prestataire, administrateur, paiement, notifications), en cohérence avec les cas d'utilisation. L'accès aux routes protégées exige une authentification et le respect du rôle de l'utilisateur : un client ne peut pas appeler une route réservée à l'administrateur, et un prestataire dont le compte n'est pas « Actif » ne peut pas se connecter. Cette couche applique les règles les plus sensibles : le verrouillage atomique d'un créneau lors d'une réservation, l'enchaînement autorisé des statuts d'une réservation, le calcul de l'acompte (30 % du tarif) et de la commission (10 % du tarif), le crédit du portefeuille du prestataire en fin de prestation et la vérification de la signature des webhooks de DexPay.
Couche 3 : Persistance (PostgreSQL)	Base de données relationnelle qui stocke les utilisateurs, justificatifs, services, créneaux, réservations, paiements, portefeuilles, retraits, avis, litiges, sanctions et notifications. Chaque classe du diagramme de conception correspond à une table, chaque association à une clé étrangère ; Laravel y accède par l'ORM Eloquent. Le verrouillage temporaire d'un créneau repose sur une opération atomique de la base : lorsque deux clients sélectionnent le même créneau au même instant, un seul obtient le verrou. Les opérations financières (enregistrement d'un paiement, crédit du portefeuille, débit lors d'un retrait) sont exécutées dans des transactions, afin que le solde d'un prestataire reste cohérent.
Couche 4 : Notifications (e-mail et push)	Appelée par la couche métier à chaque étape clé : résultat de l'examen d'un dossier, nouvelle demande de réservation, acceptation ou refus d'une demande, confirmation du paiement avec le code de réservation, décision sur un litige ou sur un avis signalé. Isolée derrière l'interface NotificationService. Deux canaux : l'e-mail (Laravel Mail via SMTP Gmail) et la notification push (Firebase Cloud Messaging). L'administrateur ne reçoit ni e-mail ni notification push : les éléments à traiter lui sont signalés par des bannières d'alerte sur le tableau de bord du back-office.
Service externe : paiement (DexPay)	Le paiement de l'acompte est confié à la passerelle DexPay (Wave ou Orange Money), isolée derrière l'interface PaiementService. Le backend crée une session de paiement et redirige le client vers la page hébergée par DexPay : l'application ne manipule donc aucune donnée bancaire. Le résultat revient au backend par un webhook signé, dont l'authenticité est vérifiée avant toute mise à jour de la réservation. DexPay est utilisée en mode test : paiements simulés, aucun argent réel échangé.
Hébergement (Dokploy)	Le backend Laravel, le back-office React et la base PostgreSQL sont déployés sur un serveur géré par Dokploy, qui récupère le code depuis le dépôt GitHub, le construit et le met en ligne en HTTPS. Les clés secrètes (clés DexPay, accès SMTP, clés Firebase) sont conservées dans les variables d'environnement du serveur et non dans le code. L'application mobile n'est pas hébergée : elle est compilée en fichier APK, mis à disposition en téléchargement pour être installé sur les téléphones Android.

9.3.3 Organisation modulaire de l'API

Les routes de l'API sont groupées en six modules, par domaine métier : authentification, client, prestataire, administrateur, paiement et notifications.

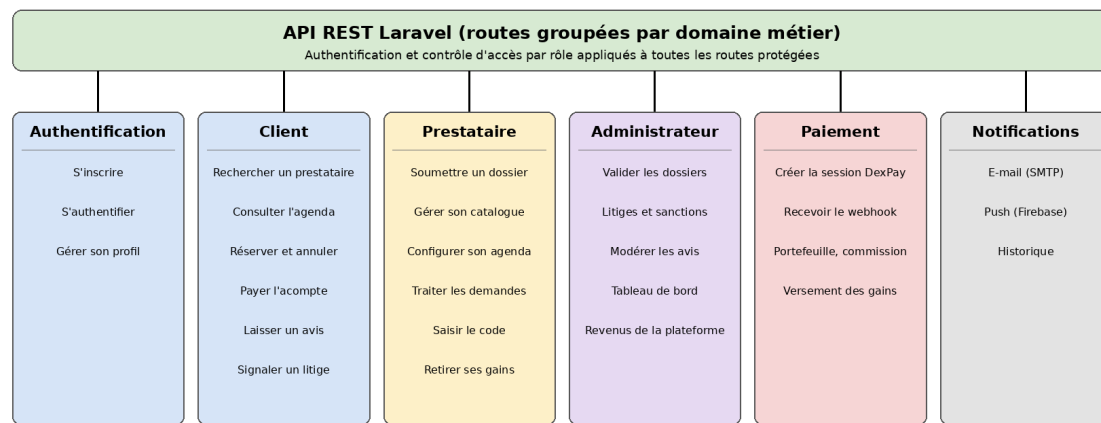


Figure 11 – Organisation modulaire de l'API par domaine métier

9.3.4 Cheminement d'une réservation

1. le client choisit un créneau dans l'agenda affiché par l'application mobile ;
2. l'application envoie la demande à l'API Laravel, qui verrouille le créneau dans PostgreSQL de façon atomique ;
3. le client confirme dans le délai du verrou : Laravel enregistre la réservation au statut « En attente », sans paiement à ce stade ;
4. Laravel déclenche une notification (push et e-mail) au prestataire ;
5. l'application affiche le récapitulatif de la réservation au client ;
6. le prestataire consulte la demande et l'accepte : Laravel passe la réservation au statut « Acceptée » (en attente de paiement) et notifie le client, qui peut alors payer l'acompte.

Si le verrou a expiré avant la confirmation, Laravel libère le créneau et renvoie le message « Délai dépassé » que l'application présente au client. Si le prestataire refuse la demande, la réservation passe à « Refusée », le créneau est libéré et le client en est informé avec le motif.

9.3.5 Cheminement du paiement et du retrait des gains

1. le client, dont la demande est « Acceptée », demande à payer l'acompte depuis l'application ;
2. Laravel lit la réservation et son tarif, puis demande à DexPay de créer une session de paiement d'un montant égal à 30 % du tarif ;
3. l'application redirige le client vers la page de paiement de DexPay, où il choisit Wave ou Orange Money et valide l'opération sur son téléphone ;
4. DexPay informe Laravel du résultat par un webhook signé ; Laravel en vérifie la signature ;
5. si le paiement est réussi, Laravel enregistre le paiement, passe la réservation à « Confirmée » et génère le code de réservation ;
6. Laravel déclenche les notifications : le client reçoit la confirmation avec son code, le prestataire est informé que la réservation est confirmée.

Si le paiement échoue, il est enregistré avec le statut « ECHOUÉ » et le client peut réessayer. Sans paiement dans un délai de 2 heures, la réservation est annulée et le créneau est libéré. À l'heure du rendez-vous, le prestataire saisit le code de réservation présenté par le client : la prestation passe à « En cours », puis à « Terminée ». À la fin de la prestation, Laravel prélève la commission de la plateforme sur le tarif et crédite le portefeuille du prestataire du reste de l'acompte ; le solde du tarif est réglé

directement entre le client et le prestataire, sur place. Le prestataire peut ensuite retirer ses gains depuis son portefeuille : Laravel débite le portefeuille, enregistre un retrait « En cours » et demande à DexPay d'effectuer le versement, dont le résultat (réussi ou échoué) est mis à jour dans l'historique du portefeuille.

10. Sécurité et protection des données

Les mesures de sécurité de la plateforme, issues des exigences et de l'architecture, sont les suivantes :

Tableau 26 – Mesures de sécurité

Domaine	Mesure
Authentification	Authentification stricte de tous les utilisateurs ; mots de passe hachés avec Bcrypt ; blocage temporaire du compte après cinq échecs consécutifs ; refus d'accès des comptes suspendus ou bannis.
Autorisation	Contrôle d'accès par rôle sur les routes protégées de l'API (client, prestataire, administrateur) ; un prestataire dont le compte n'est pas « Actif » ne peut pas se connecter.
Échanges	Toutes les requêtes passent en HTTPS (certificat géré par Dokploy), condition aussi nécessaire à la réception des webhooks de DexPay.
Paiement	Aucune donnée bancaire n'est manipulée par l'application (page de paiement hébergée par DexPay) ; webhooks signés (HMAC-SHA256) dont la signature est vérifiée avant toute mise à jour ; opérations financières exécutées dans des transactions de base de données.
Secrets	Clés DexPay, accès SMTP et clés Firebase conservés dans les variables d'environnement de Dokploy, jamais dans le code.
Confidentialité	Identité du client masquée dans les notifications au prestataire ; confidentialité des échanges lors des litiges.
Protection du framework	Protection contre les attaques courantes (CSRF, injection SQL) fournie par Laravel ; validation des données et des fichiers (justificatifs au format PDF ou image).
Sauvegarde	Sauvegarde automatique de la base de données toutes les 24 heures.

11. Matrice de traçabilité

La matrice suivante relie chaque exigence fonctionnelle aux cas d'utilisation qui la réalisent et aux règles de gestion qui la précisent, afin de vérifier que tout besoin est couvert par la modélisation.

Tableau 27 – Traçabilité des exigences fonctionnelles

Exigence	Acteur	Cas d'utilisation couvrant l'exigence	Règles de gestion
RF01	Client	S'inscrire ; Gérer son profil	RG01, RG03
RF02	Client	Rechercher un prestataire ; Filtrer les résultats	RG04
RF03	Client	Consulter l'agenda ; Réserver un créneau	RG08, RG09, RG10
RF04	Client	Suivre une réservation ; Voir le détail d'une réservation ; Annuler une réservation	RG14, RG15
RF05	Client	Consulter le profil d'un prestataire	RG04
RF06	Client	Laisser un avis	RG30
RF07	Client	Signaler un litige	RG31
RF08	Prestataire	Soumettre un dossier d'inscription ; Joindre les justificatifs ; Suivre l'état de sa validation	RG05, RG06

Exigence	Acteur	Cas d'utilisation couvrant l'exigence	Règles de gestion
RF09	Prestataire	Gérer son catalogue de services ; Configurer son agenda	—
RF10	Prestataire	Gérer ses réservations ; Traiter les demandes	RG11, RG12, RG13
RF11	Prestataire	Mettre à jour l'état de la prestation	RG14, RG22
RF12	Prestataire	Déclarer une indisponibilité	RG25, RG26
RF13	Prestataire	Consulter l'historique des prestations ; Répondre à un litige ; Envoyer une notification	RG31
RF14	Administrateur	S'authentifier ; Consulter le tableau de bord	RG01, RG02, RG03, RG35
RF15	Administrateur	Gérer la validation des comptes prestataires ; Consulter les justificatifs professionnels ; Accepter ou refuser la demande d'inscription	RG04, RG07
RF16	Administrateur	Consulter le tableau de bord (utilisateurs, statistiques des réservations, revenus de la plateforme)	RG35
RF17	Administrateur	Gérer les litiges et les sanctions ; Consulter le détail d'un litige ; Trancher le litige	RG32, RG33
RF18	Système	Envoyer une notification (inclus dans les cas d'utilisation concernés)	RG34, RG35
RF19	Administrateur	Gérer les litiges et les sanctions ; Voir l'historique des sanctions	RG01, RG33
RF20	Prestataire, Administrateur	Consulter ses avis ; Signaler un avis ; Modérer les avis signalés	RG30
RF21	Client	Payer l'acompte	RG16 à RG21, RG25
RF22	Système, Prestataire	Mettre à jour l'état de la prestation (passage à « Terminée »)	RG23, RG24, RG29
RF23	Prestataire	Retirer ses gains	RG27, RG28, RG29

12. Contraintes, limites et perspectives

12.1 Contraintes et limites du projet

- **Païement en mode test** : DexPay est utilisée dans son environnement de test (sandbox) avec des clés de test ; les transactions sont simulées et aucun argent réel n'est échangé. Cela convient à la phase de démonstration du projet.
- **HTTPS obligatoire** : l'API doit être exposée en HTTPS pour recevoir les webhooks de DexPay et sécuriser les échanges avec l'application mobile.
- **Distribution de l'application** : l'application est livrée par un fichier APK téléchargeable ; l'installation exige d'autoriser les applications provenant de sources externes.
- **Connectivité** : le système doit fonctionner correctement sur des connexions à faible débit (temps de réponse moyen inférieur à 2 secondes, images compressées sous 500 Ko).
- **Notifications e-mail** : les e-mails transitent par un serveur SMTP (Gmail) ; les notifications push par Firebase Cloud Messaging.

12.2 Perspectives

- publication de l'application sur une boutique d'applications (Google Play) ;
- passage de DexPay du mode test à l'encaissement et au versement réels ;
- changement de prestataire de paiement, sans modifier les règles métier, grâce à l'interface PaiementService ;
- ajout d'un nouveau canal de notification, grâce à l'interface NotificationService ;
- ajout de nouveaux types de services ou de nouveaux quartiers, rendu possible par la maintenabilité du code et la flexibilité de la base de données.

13. Glossaire

Tableau 28 – Glossaire

Terme	Définition
Acompte	Part du tarif (30 %) payée en ligne par le client pour confirmer sa réservation.
API REST	Interface de programmation du backend, accessible en HTTPS, échangeant des données au format JSON avec l'application mobile et le back-office.
APK	Format de fichier d'installation des applications Android.
Back-office	Application web d'administration réservée à l'administrateur.
Code de réservation	Code généré après le paiement de l'acompte, que le client présente le jour du rendez-vous et que le prestataire saisit pour démarrer la prestation.
Commission	Part prélevée par la plateforme (10 % du tarif par défaut, paramétrable) lors de la clôture d'une prestation.
Créneau	Plage horaire planifiée par un prestataire pour indiquer ses disponibilités.
Eloquent	ORM de Laravel faisant correspondre chaque classe à une table de la base de données.
HMAC-SHA256	Mécanisme de signature qui permet de vérifier l'authenticité d'un message, utilisé pour les webhooks de DexPay.
Litige	Réclamation d'un client suite à une réservation, arbitrée par l'administrateur.
Payout (versement)	Opération par laquelle DexPay reverse de l'argent à un bénéficiaire, ici les gains du prestataire.
Portefeuille	Solde des gains d'un prestataire, crédité à la fin de chaque prestation terminée.
Sandbox (mode test)	Environnement de test d'une passerelle de paiement, où les transactions sont simulées.
Session de paiement	Opération créée auprès de DexPay pour un montant donné, vers laquelle le client est redirigé pour payer.
Verrou temporaire	Blocage atomique d'un créneau pendant la réservation, qui expire s'il n'est pas confirmé.
Webhook	Appel automatique envoyé par DexPay au backend pour l'informer du résultat d'un paiement.